Q1. What is an algorithm? What are its desirable characteristics? Explain characteristics of an algorithm with the help of an example.

Answer:

Definition of an Algorithm:

The word algorithm is derived from the ninth-century mathematician Abdullah Jafar Muhammad ibn Musa Al-khowarizmi. An algorithm can be defined as:

- A set of rules for carrying out calculations either by hand or on a machine.
- A well-defined computational procedure that takes input and produces output.
- A finite sequence of instructions or steps to achieve some particular output.

Characteristics of an Algorithm:

For a good algorithm, it must satisfy the following five basic properties:

- **Input:** There must be a finite number of inputs for the algorithm.
- **Output:** There must be some output produced as a result of the execution of the algorithm.
- **Definiteness:** There must be a definite sequence of operations for the transformation of input into output.
- **Effectiveness:** Every step of the algorithm should be basic and essential.
- **Finiteness:** The transformation of input to output must be achieved in finite steps, i.e., the algorithm must stop eventually.

Desirable Characteristics:

- The algorithm should be general and able to solve several cases.
- The algorithm should use resources efficiently, taking less time and memory to produce the result.
- It should be understandable so that anyone can apply it to their own problem.
- It should follow uniqueness such that each instruction is unambiguous and clear.

Example Explaining the Characteristics:

A classic example is **Euclid's algorithm** to find the Greatest Common Divisor (GCD) of two given integers (a and b).

Explanation based on characteristics: This algorithm takes two positive numbers as **inputs** and successfully produces one **output** (the GCD). It demonstrates **definiteness** and **effectiveness** because the instructions (like checking if b=0, or dividing a by b to get remainder r) are written in basic, clear, and unambiguous steps. Finally, it strictly follows **finiteness** because the remainder r reduces in every iteration, ensuring the algorithm terminates in a finite number of steps when the condition r=0 is met.

Q2. What are the building blocks of an algorithm? Explain how to judge an algorithm, whether it is efficient or not?

Answer:

Building Blocks of an Algorithm:

An algorithm is a procedural way to write the solution to a problem and is designed using five basic building blocks:

- **Sequencing (Step-by-step actions):** A problem can be solved by performing actions in a linear sequence. The exact order of execution of these actions is critically important to ensure the correctness of the algorithm.
- **Selection (Decision):** There are situations where the execution of certain instructions depends on a given condition (a Boolean expression). Selection determines the next step to be executed.
- **Iteration (Repetition or Loop):** While solving a problem, certain actions may need to be executed multiple times or until a certain condition is met.
- **Procedure:** It is a mechanism to avoid writing the same sequence of instructions repeatedly. A sequence expected to be used multiple times is written only once outside the main algorithm, and can be called using arguments.
- **Recursion:** A procedure or algorithm which has an in-built mechanism to call itself is known as a recursive procedure.

Judging the Efficiency of an Algorithm:

An algorithm should always be both correct and efficient. The efficiency of an algorithm is defined in terms of the resource usage required to yield a correct answer. To judge an algorithm, we evaluate its **Computational Complexity** (how hard it is to execute the algorithm based on the input size). This is evaluated using two primary measures:

- **Time Complexity:** This is measured by counting the number of key operations (like the number of comparisons in sorting/searching) executed for a given input. It represents the running time required by the algorithm to generate the output. An algorithm is judged to be better if it takes less running time.
- **Space/Memory Complexity:** This is measured by determining the maximum amount of memory storage required by the algorithm during its execution.

Since execution performance heavily depends on how the input data looks, efficiency is usually judged by theoretically calculating the **Worst-case** (maximum steps taken), **Best-case** (minimum steps taken), and **Average-case** performance using asymptotic bounds like Big-Oh (O), Omega (Ω), and Theta (Θ).

Q3. What is Euclid's algorithm to find GCD of two given integers? Write the steps involved in finding GCD of (a, b) using Euclid's algorithm.

Answer:

What is Euclid's Algorithm?

Euclid's algorithm is a highly efficient and classical mathematical method used to find the **Greatest Common Divisor (GCD)** of two given non-negative integers. The GCD of two integers is the largest positive integer that perfectly divides both numbers without leaving any remainder.

Steps involved in finding GCD of (a, b):

According to the algorithmic approach, Euclid's method relies on continuous division. Here are the exact steps to find the GCD of two numbers a and b (assuming a \ge b):

- **Step 1:** Check the value of b. If b = 0, then the algorithm terminates, and the GCD is a.
- **Step 2:** If b \neq 0, divide a by b and find the remainder r. (Mathematically, r = a \pmod b).
- **Step 3:** Update the variables by swapping their values: Assign the value of b to a (i.e., a = b), and assign the value of the remainder r to b (i.e., b = r).
- **Step 4:** Go back to Step 1 and repeat this process until b becomes exactly 0.

Brief Example for Clarity: To find the GCD of 48 and 18:

- **Iteration 1:** a = 48, b = 18. Remainder $r = 48 (mod 18) = 12$. New values: a = 18, b = 12.
- **Iteration 2:** a = 18, b = 12. Remainder $r = 18 (mod 12) = 6$. New values: a = 12, b = 6.
- **Iteration 3:** a = 12, b = 6. Remainder $r = 12 (mod 6) = 0$. New values: a = 6, b = 0.
- Now b = 0, so the algorithm stops. The GCD is a, which is **6**.

Q4. What are Optimization and Decision problems? Give an example of each. Formulate Traveling salesperson problem & Graph coloring problem as optimization and decision problems.

Answer:

Optimization and Decision Problems:

- **Optimization Problem:** These are computational problems where there are multiple feasible (valid) solutions, and the primary goal is to find the **"best"** or **optimal solution** among them. The optimal solution usually means either minimizing a value (like cost, time, or distance) or maximizing a value (like profit).
  - *Example:* Finding the shortest possible route (minimum distance) between two cities on a map.
- **Decision Problem:** These are problems that demand a strict, binary answer: either **"Yes" or "No"**. They do not ask for the best solution; instead, they verify whether a specific condition or constraint is met.
  - *Example:* Does there exist a route between two cities with a total distance of 50 km or less? (The answer is strictly Yes or No).

Formulation of the Traveling Salesperson Problem (TSP):

In TSP, a salesperson must start from a home city, visit every other city exactly once, and return to the home city.

- **As an Optimization Problem:** Find a valid tour that visits every city exactly once and returns to the starting point such that the total traveling distance (or cost) is **minimized**.
- **As a Decision Problem:** Given a specific integer boundary K, does there exist a valid tour visiting all cities exactly once with a total traveling distance that is **less than or equal to K (\le K)**?

Formulation of the Graph Coloring Problem:

In Graph Coloring, colors must be assigned to the vertices (nodes) of a graph such that no two adjacent vertices (nodes connected directly by an edge) share the same color.

- **As an Optimization Problem:** Find the **minimum number of colors** (known as the chromatic number) required to properly color the entire graph without any conflicts.
- **As a Decision Problem:** Given a fixed integer m, can the graph be properly colored using **exactly m or fewer colors** such that no two adjacent vertices have the same color?

Q5. "The best-case analysis is not as important as the worst-case analysis of an algorithm." Yes or No. Justify your answer with the help of an example.

Answer:

Yes, the statement is absolutely correct. The best-case analysis is generally not as important as the worst-case analysis of an algorithm.

Justification:

1. **Guaranteed Upper Bound:** The worst-case running time of an algorithm gives us an upper bound on the execution time for any input. It provides a guarantee that the algorithm will never take any longer than this specified time. This is crucial for real-time systems where strict deadlines must be met.
2. **Frequency of Occurrence:** The worst-case scenario occurs fairly often in real-world applications (e.g., searching for an item that is not present in a database). In contrast, the best-case scenario is a rare "lucky" situation that occurs under highly specific conditions.
3. **Average Case Proximity:** The average-case running time is often much closer to the worst-case running time than to the best-case running time. Therefore, optimizing for the worst-case indirectly improves the overall robustness of the system.

Example:

Consider the **Linear Search** algorithm, which searches for a specific element in an array of n elements.

- **Best-Case:** The required element is found at the very first position (index 0). The time taken is O(1). While this is very fast, we cannot build a reliable software system expecting the user's data to always be at the first position.
- **Worst-Case:** The required element is located at the very last position of the array, or it is not present in the array at all. The algorithm must check every single element, taking O(n) time.

If a database has 1 million records, knowing the worst-case (O(n)) allows the developer to allocate enough server time and memory to handle checking all 1 million records, making the worst-case analysis far more important and practical.

Q6. What are asymptotic notations? Explain any two asymptotic notations with suitable example for each.

Answer:

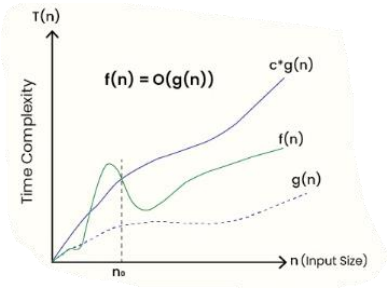
What are Asymptotic Notations?

Asymptotic notations are mathematical tools used to define the performance and complexity (time or space) of an algorithm in terms of the input size (n). They help in analyzing how the runtime of an algorithm grows as the input size grows toward infinity, completely ignoring hardware-specific constants and lower-order terms.

Two Asymptotic Notations with Examples:

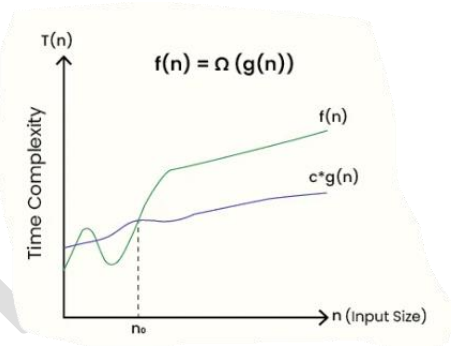
A) Big Oh Notation (O):

- **Definition:** It provides the **Asymptotic Upper Bound** of an algorithm. It represents the maximum time required by an algorithm for all possible inputs, essentially defining the worst-case time complexity.
- **Mathematical representation:** A function $f(n) = O(g(n))$ if there exist positive constants c and n_0 such that $0 \leq f(n) \leq c \cdot g(n)$ for all $n \geq n_0$.
- **Example:** Let $f(n) = 3n + 5$. Here, $3n + 5 \leq 4n$ for all $n \geq 5$. Therefore, $f(n) = O(n)$.
- **Algorithm Example:** The worst-case time complexity of Linear Search is $O(n)$.



B) Big Omega Notation (Ω):

- **Definition:** It provides the **Asymptotic Lower Bound** of an algorithm. It represents the minimum time required by an algorithm for all possible inputs, essentially defining the best-case time complexity.
- **Mathematical representation:** A function $f(n) = \Omega(g(n))$ if there exist positive constants c and n_0 such that $0 \leq c \cdot g(n) \leq f(n)$ for all $n \geq n_0$
- **Example:** Let $f(n) = 3n + 5$. Here, $3n + 5 \geq 3n$ for all $n \geq 1$. Therefore, $f(n) = \Omega(n)$.
- **Algorithm Example:** The best-case time complexity of Linear Search (when the element is found at the first index) is $\Omega(1)$.

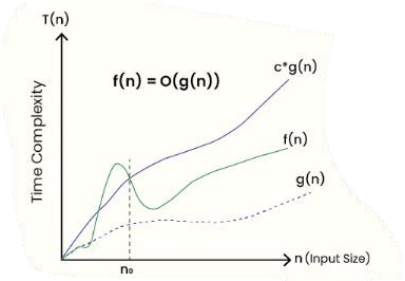


Q7. Write a mathematical definition of O (Big Oh). Assume that the function $f(n) = 2n^2 + 3n + 1$. Show that $f(n) = O(n^2)$.

Answer:

Mathematical Definition of Big-Oh (O): Big-Oh notation is used to define the asymptotic upper bound of a function. Mathematically, a function $f(n)$ is said to be $O(g(n))$, written as $f(n) = O(g(n))$, if there exist strictly positive constants c and n_0 such that:

$$0 \leq f(n) \leq c \cdot g(n) \quad \text{for all } n \geq n_0$$



Proof to show $f(n) = O(n^2)$: Given function: $f(n) = 2n^2 + 3n + 1$ Target: We need to find constants c and n_0 such that $2n^2 + 3n + 1 \leq c \cdot n^2$ for all $n \geq n_0$.

Let's assume $n \geq 1$. For all $n \geq 1$, the following inequalities hold true:

- $n \leq n^2$
- $1 \leq n^2$

Substitute these into our original function: $f(n) = 2n^2 + 3n + 1$ $f(n) \leq 2n^2 + 3(n^2) + 1(n^2)$ $f(n) \leq 6n^2$

Here, we have successfully found our constants:

- $c = 6$
- $n_0 = 1$

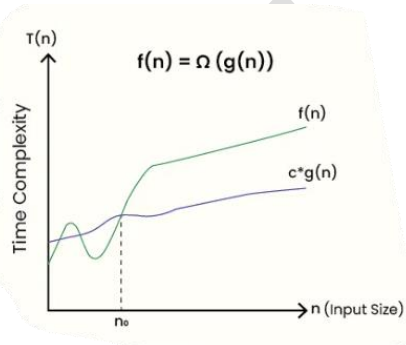
Since $2n^2 + 3n + 1 \leq 6n^2$ for all $n \geq 1$, it satisfies the definition of Big-Oh. **Hence proved, $f(n) = O(n^2)$.**

Q8. Write a mathematical definition of Big Omega (Ω). For the functions defined by $f(n) = 3n^3 + 2n^2 + 1$ and $g(n) = 2n^2 + 3$, verify that $f(n) = \Omega(g(n))$.

Answer:

Mathematical Definition of Big-Omega (Ω): Big-Omega notation provides the asymptotic lower bound of a function. Mathematically, a function $f(n)$ is said to be $\Omega(g(n))$, written as $f(n) = \Omega(g(n))$, if there exist strictly positive constants c and n_0 such that:

$$0 \leq c \cdot g(n) \leq f(n) \quad \text{for all } n \geq n_0$$



Verification that $f(n) = \Omega(g(n))$: Given functions:

- $f(n) = 3n^3 + 2n^2 + 1$
- $g(n) = 2n^2 + 3$

Target: We need to find positive constants c and n_0 such that $c \cdot (2n^2 + 3) \leq 3n^3 + 2n^2 + 1$ for all $n \geq n_0$.

Let's analyze $f(n)$ for $n \geq 1$: $f(n) = 3n^3 + 2n^2 + 1$ Since all terms are positive, dropping the smaller terms makes the function smaller. Therefore: $f(n) \geq 3n^3$

Now, let's analyze $g(n)$ for $n \geq 1$: $g(n) = 2n^2 + 3$ Since $3 \leq 3n^2$ for $n \geq 1$: $g(n) \leq 2n^2 + 3n^2$ $g(n) \leq 5n^2$

Now we want to establish: $c \cdot 5n^2 \leq 3n^3$. Dividing both sides by n^2: $5c \leq 3n$. Let's choose $c = 1$. Then we need $5 \leq 3n$, which is true for all $n \geq 2$.

So, for $c = 1$ and $n_0 = 2$: $1 \cdot g(n) \leq 5n^2 \leq 3n^3 \leq f(n)$ $1 \cdot (2n^2 + 3) \leq 3n^3 + 2n^2 + 1$

Since we found $c = 1$ and $n_0 = 2$ satisfying the condition $c \cdot g(n) \leq f(n)$, **Hence verified,** $f(n) = \Omega(g(n))$**.**

Q9. What are Big 'O' and Big 'Ω' notations? Explain with the help of a representative diagram.

Answer:

Big 'O' (Big Oh) Notation:

- **Definition:** Big 'O' notation is a mathematical tool used to define the **Asymptotic Upper Bound** of an algorithm. It represents the maximum amount of time an algorithm can possibly take to execute for a given input size (i.e., the **Worst-Case Time Complexity**). It mathematically guarantees that the algorithm will never take longer than this defined limit.
- **Mathematical representation:** A function $f(n)$ is said to be $O(g(n))$ if there exist positive constants c and n_0 such that:

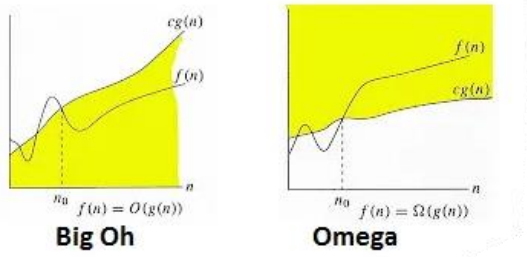
$$f(n) \leq c \cdot g(n) \quad \text{for all } n \geq n_0$$

Big 'Ω' (Big Omega) Notation:

- **Definition:** Big 'Ω' notation defines the **Asymptotic Lower Bound** of an algorithm. It represents the minimum amount of time an algorithm will require to execute (i.e., the **Best-Case Time Complexity**). It indicates that the algorithm will take at least this much time, regardless of the input conditions.
- **Mathematical representation:** A function $f(n)$ is said to be $\Omega(g(n))$ if there exist positive constants c and n_0 such that:

$$0 \leq c \cdot g(n) \leq f(n) \quad \text{for all } n \geq n_0$$

Representative Diagram: The diagram below represents both notations simultaneously. Here, $f(n)$ represents the actual running time of the algorithm. The upper curve $c_2 \cdot g(n)$ represents the Big 'O' (upper bound limit), and the lower curve $c_1 \cdot g(n)$ represents the Big 'Ω' (lower bound limit) for all input sizes strictly greater than n_0.



Q10. Arrange the following growth rates in increasing order: $O(n^2)$, $O(2^n)$, $O(n\log n)$, $O(2!)$, $O(1)$, $O(\log n)$.

Answer:

To arrange the growth rates in **increasing order**, we must place the fastest executing (least time-consuming) time complexities first, and the slowest executing (most time-consuming) time complexities last.

Let's carefully analyze the given terms:

1. $O(1)$: This is **Constant Time**. The execution time remains exactly the same regardless of how large the input size n gets. This is the absolute fastest complexity.
2. $O(2!)$: The mathematical value of 2! (2 factorial) is exactly $2 \times 1 = 2$. Since 2 is just a fixed constant number, $O(2!)$ is completely identical and equivalent to $O(1)$.
3. $O(\log n)$: This is **Logarithmic Time**. It is slightly slower than constant time but highly efficient for large inputs (e.g., used in Binary Search).
4. $O(n\log n)$: This is **Linearithmic Time**. It is slower than $O(\log n)$ and normal linear time $O(n)$, but it is much faster than quadratic time (e.g., used in Merge Sort and Quick Sort).
5. $O(n^2)$: This is **Quadratic Time**. The execution time grows significantly as the input increases (e.g., used in Bubble Sort).

6. $O(2^n)$: This is **Exponential Time**. It is the slowest among all the given options; even a very small increase in input size causes a massive, unmanageable increase in execution time.

Correct Increasing Order:

$$O(1) = O(2!) < O(\log n) < O(n\log n) < O(n^2) < O(2^n)$$

(Note: $O(1)$ and $O(2!)$ are placed together with an equal sign at the beginning because they both represent constant time).

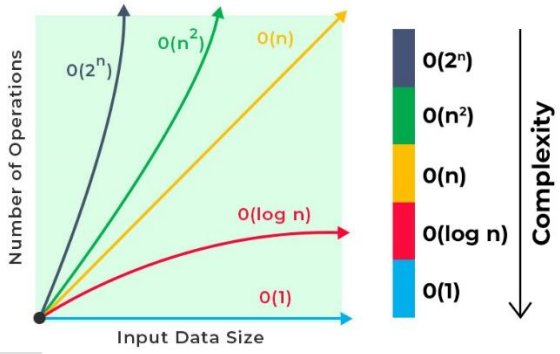
Q11. Whether the following is in correct order? $1, \log n, n, n\log n, n^2, 2^n, n!$

Answer:

Yes, the given sequence of growth rates is in the **correct increasing order** (from the fastest execution time to the slowest execution time).

Detailed Explanation of the Order: To understand why this is correct, we compare how the execution time grows as the input size (n) becomes very large:

1. 1 **(Constant Time - $O(1)$):** The execution time is fixed and does not depend on the input size at all. It is the fastest.
2. $\log n$ **(Logarithmic Time - $O(\log n)$):** The execution time grows very slowly even if the input size increases exponentially (e.g., dividing the search space in half, like in Binary Search).
3. n **(Linear Time - $O(n)$):** The execution time grows directly in proportion to the input size.
4. $n\log n$ **(Linearithmic Time - $O(n\log n)$):** This grows slightly faster than linear time. It is the standard best-case and average-case time complexity for highly efficient comparison-based sorting algorithms like Merge Sort and Quick Sort.
5. n^2 **(Quadratic Time - $O(n^2)$):** The execution time grows in proportion to the square of the input size (usually caused by two nested loops).
6. 2^n **(Exponential Time - $O(2^n)$):** The execution time doubles with every single addition to the input size. This is extremely slow and generally impractical for large inputs.
7. $n!$ **(Factorial Time - $O(n!)$):** This represents the number of ways to arrange n items. For any $n \geq 4$, $n!$ grows much faster than 2^n. It is the absolute slowest growth rate commonly encountered in algorithm analysis (e.g., generating all permutations of a string or naive Traveling Salesperson Problem).



Q12. Calculate the time complexity of the following program fragments using Big Oh notation:

(i) For (i=0; i<n; i++) a[i]=0; (ii) For (i=1; i<=n; i=i*2) { x=x+i; } (iii) for(i=0; i<n; i++) for (j=0; j<n; j++) A[i]=A[i]+A[j];

Answer:

Calculation for Fragment (i):

```
For (i=0; i<n; i++)
  a[i]=0;
```

- **Analysis:** The loop initializes the variable i to 0 and increments it by 1 (i++) during each iteration until it reaches $n-1$.
- **Total Iterations:** The loop executes exactly n times. The operation inside the loop (a[i]=0) takes constant time $O(1)$.
- **Time Complexity:** $n \times O(1) = O(n)$

Calculation for Fragment (ii):

```
For (i=1; i<=n; i=i*2) {
   x=x+i;
}
```

- **Analysis:** The loop variable i starts at 1. However, instead of incrementing by 1, it is **multiplied by** 2 in each iteration (i=i*2).
- **Progression of i:** The values of i will be 1,2,4,8,16, … , 2^k.
- **Total Iterations:** The loop terminates when $2^k > n$. Solving for k gives $k = \log_2 n$. Therefore, the loop runs $\log_2 n$ times.
- **Time Complexity:** $O(\log n)$

Calculation for Fragment (iii):

```
for(i=0; i<n; i++)
   for (j=0; j<n; j++)
      A[i]=A[i]+A[j];
```

- **Analysis:** This fragment contains **nested loops**.
- **Outer Loop:** The i loop runs from 0 to $n-1$, which is exactly n iterations.
- **Inner Loop:** For *every single iteration* of the outer loop, the inner j loop runs from 0 to $n-1$, which is another n iterations.
- **Total Iterations:** Since the inner loop executes n times for each of the n iterations of the outer loop, the total number of operations is $n \times n = n^2$.
- **Time Complexity:** $O(n^2)$

Q13. List one algorithm each for the following time complexities: (i) $O(m\log n)$ (ii) $O(\log n)$ (iii) $O(n^2)$.

Answer:

Here are standard algorithms that correspond to the given worst-case time complexities:

(i) $O(m\log n)$**:** * **Algorithm: Kruskal's Algorithm** (for finding the Minimum Spanning Tree of a graph).

- **Explanation:** In a graph with m edges and n vertices, Kruskal's algorithm first sorts all the edges, which takes $O(m\log m)$ time. Since $m \leq n^2$, $\log m \leq 2\log n$, so sorting takes $O(m\log n)$. The subsequent Union-Find operations take $O(m\log n)$ time. Thus, the overall time complexity is $O(m\log n)$.

(ii) $O(\log n)$**:**

- **Algorithm: Binary Search** (for searching an element in a sorted array).
- **Explanation:** Binary search repeatedly divides the search interval in half. If the array size is n, it takes at most $\log_2 n$ steps to find the target element or conclude it is missing, resulting in an $O(\log n)$ time complexity.

(iii) $O(n^2)$**:**

- **Algorithm: Bubble Sort** (or Selection Sort / Insertion Sort).
- **Explanation:** To sort an array of n elements, Bubble sort uses two nested loops. The outer loop runs n times, and for each pass, the inner loop runs up to n times to swap adjacent unsorted elements. This results in $n \times n = n^2$ operations, giving a time complexity of $O(n^2)$.

Q14. Using mathematical induction, prove that the sum of first 'n' positive integers is $(n(n+1)/2)$.

Answer:

Principle of Mathematical Induction: To prove a mathematical statement $P(n)$ for all positive integers n, we must complete two steps:

7. **Base Case:** Prove that $P(1)$ is true.
8. **Inductive Step:** Assume $P(k)$ is true (Inductive Hypothesis), and use this assumption to prove that $P(k+1)$ is also true.

Proof:

Let $P(n)$ be the statement that the sum of the first n positive integers is given by the formula:

$$1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}$$

Step 1: Base Case Let us test the formula for $n = 1$ (the first positive integer).

- **Left Hand Side (LHS):** The sum of the first 1 integer is simply 1.
- **Right Hand Side (RHS):** Substitute $n = 1$ into the formula:

$$\text{RHS} = \frac{1(1+1)}{2} = \frac{1(2)}{2} = \frac{2}{2} = 1$$

Since LHS = RHS $(1 = 1)$, the base case $P(1)$ is **True**.

Step 2: Inductive Hypothesis Assume that the statement $P(n)$ is true for some arbitrary positive integer k. Therefore, we assume the following equation is completely true:

$$1 + 2 + 3 + \cdots + k = \frac{k(k+1)}{2} \quad \text{— (Equation 1)}$$

Step 3: Inductive Step Now, we must prove that the statement holds true for $n = k + 1$. That means we need to prove:

$$1 + 2 + 3 + \cdots + k + (k+1) = \frac{(k+1)\big((k+1)+1\big)}{2} = \frac{(k+1)(k+2)}{2}$$

Let us take the Left Hand Side (LHS) for $n = k + 1$:

$$\text{LHS} = (1 + 2 + 3 + \cdots + k) + (k+1)$$

Substitute the value of $(1 + 2 + \cdots + k)$ from our Inductive Hypothesis (Equation 1):

$$\text{LHS} = \left[\frac{k(k+1)}{2}\right] + (k+1)$$

Now, take $(k + 1)$ as a common factor:

$$\text{LHS} = (k+1)\left[\frac{k}{2} + 1\right]$$

Take the Least Common Multiple (LCM) inside the bracket:

$$\text{LHS} = (k+1)\left[\frac{k+2}{2}\right]$$

$$\text{LHS} = \frac{(k+1)(k+2)}{2}$$

This exactly matches our target Right Hand Side (RHS) for $n = k + 1$.

Conclusion: Since $P(1)$ is true, and assuming $P(k)$ is true implies that $P(k + 1)$ is also true, then by the Principle of Mathematical Induction, the formula $1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}$ is mathematically proven to be true for all positive integers n.

Q15. Use mathematical induction to prove that: $\sum i^2 = \frac{n(n+1)(2n+1)}{6}$.

Answer:

Objective: We need to prove the statement P(n) for all positive integers n, where P(n) is:

$$1^2 + 2^2 + 3^2 + \cdots + n^2 = \frac{n(n+1)(2n+1)}{6}$$

Step 1: Base Case First, we test if the formula holds true for n = 1.

- **Left Hand Side (LHS):** The sum of the square of the first integer is $1^2 = 1$
- **Right Hand Side (RHS):** Substitute n = 1 into the formula:

$$\text{RHS} = \frac{1(1+1)(2(1)+1)}{6} = \frac{1(2)(3)}{6} = \frac{6}{6} = 1$$

Since LHS = RHS, the base case P(1) is **True**.

Step 2: Inductive Hypothesis We assume that the statement P(n) is true for some arbitrary positive integer k. Thus, we assume the following equation is completely valid:

$$1^2 + 2^2 + 3^2 + \cdots + k^2 = \frac{k(k+1)(2k+1)}{6} \text{— (Equation 1)}$$

Step 3: Inductive Step Now, we must prove that the formula is also true for n = k+1. The target equation to prove is:

$$1^2 + 2^2 + \cdots + k^2 + (k+1)^2 = \frac{(k+1)((k+1)+1)(2(k+1)+1)}{6} = \frac{(k+1)(k+2)(2k+3)}{6}$$

Let's evaluate the Left Hand Side (LHS) for n = k+1:

$$\text{LHS} = (1^2 + 2^2 + \cdots + k^2) + (k+1)^2$$

Substitute the assumed value from Equation 1 into the LHS:

$$\text{LHS} = [\frac{k(k+1)(2k+1)}{6}] + (k+1)^2$$

Take (k+1) as a common factor from both terms:

$$\text{LHS} = (k+1)[\frac{k(2k+1)}{6} + (k+1)]$$

Find the common denominator (6) inside the bracket:

$$\text{LHS} = (k+1)[\frac{2k^2 + k + 6(k+1)}{6}]$$

$$\text{LHS} = (k+1)[\frac{2k^2 + k + 6k + 6}{6}]$$

$$\text{LHS} = (k+1)[\frac{2k^2 + 7k + 6}{6}]$$

Now, factorize the quadratic equation $2k^2 + 7k + 6$ by splitting the middle term (7k = 4k + 3k):

$$2k^2 + 4k + 3k + 6 = 2k(k+2) + 3(k+2) = (2k+3)(k+2)$$

Substitute this factored form back into the LHS:

$$\text{LHS} = \frac{(k+1)(k+2)(2k+3)}{6}$$

This exactly matches our target RHS for n = k+1.**Conclusion:** By the Principle of Mathematical Induction, the formula is proved to be true for all positive integers n.

Q16. Prove that for all non-negative integers n: $2^0 + 2^1 + 2^2 + \cdots + 2^n = 2^{n+1} - 1$.

Answer:

Objective: We need to prove the statement P(n) for all **non-negative** integers n. *(Note: Non-negative integers start from 0, so our base case will be n=0 instead of n=1)*. The statement $2^0 + 2^1 + 2^2 + \cdots + 2^n = 2^{n+1} - 1$

Step 1: Base Case Let us test the formula for n = 0.

- **Left Hand Side (LHS):** The first term in the series is $2^0 = 1$.
- **Right Hand Side (RHS):** Substitute n = 0 into the formula:

$$\text{RHS} = 2^{0+1} - 1 = 2^1 - 1 = 2 - 1 = 1$$

Since LHS = RHS (1 = 1), the base case P(0) is **True**.

Step 2: Inductive Hypothesis Assume that the statement P(n) is true for some arbitrary non-negative integer k. Therefore, we assume the following is true:

$$2^0 + 2^1 + 2^2 + \cdots + 2^k = 2^{k+1} - 1 \text{— (Equation 1)}$$

Step 3: Inductive Step We must now prove that the statement holds true for n = k+1. The target we want to reach is:

$$2^0 + 2^1 + \cdots + 2^k + 2^{k+1} = 2^{(k+1)+1} - 1 = 2^{k+2} - 1$$

Let's evaluate the Left Hand Side (LHS) for n = k+1:

$$\text{LHS} = (2^0 + 2^1 + \cdots + 2^k) + 2^{k+1}$$

Substitute the assumed value from our Inductive Hypothesis (Equation 1) into the LHS:

$$\text{LHS} = (2^{k+1} - 1) + 2^{k+1}$$

Rearrange the terms:

$$\text{LHS} = 2^{k+1} + 2^{k+1} - 1$$

Notice that adding the same term twice is the same as multiplying it by 2 (e.g., x + x = 2x). Therefore:

$$\text{LHS} = 2 \cdot (2^{k+1}) - 1$$

Using the laws of exponents ($x^a \cdot x^b = x^{a+b}$), we can combine the terms:

$$\text{LHS} = 2^1 \cdot 2^{k+1} - 1$$

$$\text{LHS} = 2^{(k+1)+1} - 1$$

$$\text{LHS} = 2^{k+2} - 1$$

This perfectly matches our target Right Hand Side (RHS) for n = k+1.**Conclusion:** Since P(0) is true, and assuming P(k) is true leads to P(k+1) being true, the statement is mathematically proved for all non-negative integers n using Mathematical Induction.

Q17. What is polynomial evaluation? What are its methods of evaluation? Evaluate $P(x) = 3x^2 + 5x + 6$ using Horner's rule at x=3.

Answer:

What is Polynomial Evaluation?

A polynomial of degree n is a mathematical expression of the form:

$$P(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0$$

Polynomial evaluation is the computational process of finding the numerical value of this polynomial expression when a specific numerical value is substituted for the variable x.

Methods of Evaluation:

There are generally two algorithmic methods to evaluate a polynomial:

- **The Naïve / Direct Method:** This method calculates the value of each term independently from scratch. For a term like $a_k x^k$, it computes x^k by multiplying x by itself k times, and then multiplies it by a_k. This is highly inefficient and takes $\boldsymbol{O(n^2)}$ time complexity in its worst brute-force implementation.
- **Horner's Rule (or Horner's Method):** This is a highly optimized algorithm that evaluates the polynomial by completely avoiding the direct computation of high powers of x. It repeatedly factors out x from the polynomial to create nested additions and multiplications. It is highly efficient, reducing the time complexity to exactly **O(n)** using only n multiplications and n additions.

Evaluate $P(x) = 3x^2 + 5x + 6$ using Horner's rule at x=3:

- **Step 1: Rewrite using Horner's Rule:** Factor out x from the first two terms:

$$P(x) = x(3x + 5) + 6$$

- **Step 2: Substitute x = 3 and calculate from the innermost parenthesis to the outside:**

$$P(3) = 3 \cdot (3(3) + 5) + 6$$

  - Inner bracket multiplication: $3 \times 3 = 9$
  - Inner bracket addition: $9 + 5 = 14$
  - Outer multiplication: $3 \times 14 = 42$
  - Outer addition: $42 + 6 = 48$
- **Final Answer:** The value of the polynomial at x=3 is **48**.

Q18. Apply Horner's method for evaluating a polynomial expression $\boldsymbol{p(x) = 6x^6 + 5x^5 + 4x^4 - 3x^3 + 8x - 7}$ at x = 3. Calculate: (i) How many times will the loop execute? (ii) What will be the total number of multiplication and addition operations?

Answer:

Applying Horner's Method:

First, observe the given polynomial. Notice that the **x^2 term is missing**, which means its coefficient is 0. We MUST include it for the algorithm to work correctly.

$$p(x) = 6x^6 + 5x^5 + 4x^4 - 3x^3 + 0x^2 + 8x - 7$$

- **Coefficients:** $a_6 = 6$, $a_5 = 5$, $a_4 = 4$, $a_3 = -3$, $a_2 = 0$, $a_1 = 8$, $a_0 = -7$
- **Value of x:** 3

Let's trace the standard Horner's algorithm loop (Initialize result = a_n):

- **Initialization:** result = a_6 = 6
- **Iteration 1 (for a_5):** result = $(6 \times 3) + 5 = 18 + 5 =$
- **Iteration 2 (for a_4):** result = $(23 \times 3) + 4 = 69 + 4 =$
- **Iteration 3 (for a_3):** result = $(73 \times 3) + (-3) = 219 - 3 =$
- **Iteration 4 (for a_2):** result = $(216 \times 3) + 0 = 648 + 0 =$
- **Iteration 5 (for a_1):** result = $(648 \times 3) + 8 = 1944 + 8 =$
- **Iteration 6 (for a_0):** result = $(1952 \times 3) + (-7) = 5856 - 7 =$

Final Evaluated Value: 5849

Calculations:

(i) How many times will the loop execute?

In Horner's algorithm, for a polynomial of degree n, the loop executes exactly n times to process the remaining n coefficients.

Here, the degree of the polynomial is n = 6.

Therefore, **the loop will execute exactly 6 times**.

(ii) What will be the total number of multiplication and addition operations?

The primary mathematical advantage of Horner's method is its strict operational count. For a polynomial of degree n, the algorithm guarantees exactly n multiplications and n additions.

Since n = 6:

- **Total number of multiplications: 6**
- **Total number of additions (including subtractions): 6**

(You can verify this by counting the operations in the step-by-step trace above).

Q19. Compute x^{283} by using left-to-right binary exponentiation.

Answer:

What is Left-to-Right Binary Exponentiation? Binary exponentiation (also known as exponentiation by squaring) is a highly efficient algorithm to compute x^n in $O(\log n)$ time, rather than the naïve $O(n)$ time. The left-to-right method uses the binary representation of the exponent n and reads the bits from the Most Significant Bit (MSB, leftmost) to the Least Significant Bit (LSB, rightmost).

The Algorithm Rules:

- Initialize result = 1.
- Traverse the binary bits of the exponent from left to right.
- For **every** bit, square the result: result = result * result.
- If the current bit is **1**, also multiply by the base: result = result * x.

Step-by-Step Computation for x^{283}:

- **Step 1: Convert the exponent (283) into Binary:**
  - $283 \div 2 = 141$ (Remainder **1**)
  - $141 \div 2 = 70$ (Remainder **1**)
  - $70 \div 2 = 35$ (Remainder **0**)
  - $35 \div 2 = 17$ (Remainder **1**)
  - $17 \div 2 = 8$ (Remainder **1**)
  - $8 \div 2 = 4$ (Remainder **0**)
  - $4 \div 2 = 2$ (Remainder **0**)
  - $2 \div 2 = 1$ (Remainder **0**)
  - $1 \div 2 = 0$ (Remainder **1**)
  - Reading remainders from bottom to top, **283 in binary is:** 100011011_2
- **Step 2: Apply the Left-to-Right Rules:**

  Let's trace the algorithm using the binary string 1 0 0 0 1 1 0 1 1. We start with result = 1.

Bit (Left to Right)	Operation 1: Square the Result (result = result^2)	Operation 2: If Bit == 1, Multiply by base (result = result * x)	Final Result at this Step
1 (MSB)	$1^2 = 1$	$1 \times x = x$	x^1
0	$(x^1)^2 = x^2$	*(Skip, since bit is 0)*	x^2
0	$(x^2)^2 = x^4$	*(Skip, since bit is 0)*	x^4
0	$(x^4)^2 = x^8$	*(Skip, since bit is 0)*	x^8

1	$(x^8)^2 = x^{16}$	$x^{16} \times x = x^{17}$	x^{17}
1	$(x^{17})^2 = x^{34}$	$x^{34} \times x = x^{35}$	x^{35}
0	$(x^{35})^2 = x^{70}$	*(Skip, since bit is 0)*	x^{70}
1	$(x^{70})^2 = x^{140}$	$x^{140} \times x = x^{141}$	x^{141}
1 (LSB)	$(x^{141})^2 = x^{282}$	$x^{282} \times x = x^{283}$	x^{283}

Conclusion: Using this method, x^{283} is successfully computed using only **8 squarings and 5 multiplications** (total 13 operations), which is vastly faster than multiplying x by itself 282 times!

Q20. Give a recursive function to find the height of a binary tree. What is the running time of this algorithm?

Answer:

Recursive Function to Find the Height: The height of a binary tree is defined as the number of edges on the longest downward path between the root and a leaf node. If the tree is empty, its height is usually considered -1. If it has only one node (the root), its height is 0.

The recursive approach follows a simple logic: The height of a tree is equal to the maximum of the heights of its left sub-tree and right sub-tree, plus 1 (for the edge connecting the root to the sub-trees).

C/C++ Recursive Code:

```
// Definition of the tree node
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

// Recursive function to calculate height
int findHeight(struct Node* root) {
    // Base Case: If the tree is empty, return -1
    if (root == NULL) {
        return -1;
    }

    // Recursive Case: Find height of left and right sub-trees
    int leftHeight = findHeight(root->left);
    int rightHeight = findHeight(root->right);

    // Return the maximum of the two heights, plus 1 for the current node's edge
    if (leftHeight > rightHeight) {
        return leftHeight + 1;
    } else {
        return rightHeight + 1;
    }
}
```

Running Time of the Algorithm:

- To calculate the height of the tree, the algorithm must visit **every single node** exactly once to check if it has children and to calculate the depth of that specific path.
- Let n be the total number of nodes in the binary tree.
- Since each node is visited exactly once, and the work done at each node (comparing two integers and adding 1) takes constant $O(1)$ time, the total running time is proportional to n.
- **Time Complexity:** $O(n)$ (Linear Time).
- **Space Complexity (Auxiliary Stack Space):** The space complexity is determined by the maximum depth of the recursion tree. In the worst-case scenario (a highly unbalanced, skewed tree), it will be $O(n)$. In the best-case scenario (a perfectly balanced tree), the stack depth will be $O(\log n)$.

Q21. Explain the use of master method. Write and interpret all the three cases of the master method to solve recurrence relation problem.

Answer:

Use of the Master Method: The Master Method provides a direct, "cookbook" style formula for solving recurrence relations that arise from **Divide and Conquer** algorithms. Instead of drawing complex recurrence trees or making mathematical guesses, the Master Method allows us to find the asymptotic time complexity (Big-Θ) almost instantly for recurrences of the standard form:

$$T(n) = aT(n/b) + f(n)$$

Where:

- n is the size of the problem.
- $a \geq 1$ is the number of subproblems in the recursion.
- $b > 1$ is the factor by which the subproblem size is reduced.
- $f(n)$ is the cost of the work done outside the recursive calls (usually dividing the problem and merging the solutions).

The Three Cases of the Master Method: To use the Master Method, we always compare the function $f(n)$ with the mathematical term $n^{\log_b a}$. The relationship between these two determines which case applies:

- **Case 1: The leaves dominate the tree (Recursive work is heavier)**
  - *Condition:* If $f(n) = O\left(n^{\log_b a - \epsilon}\right)$ for some constant $\epsilon > 0$.
  - *Interpretation:* The work done at the root $f(n)$ is polynomially smaller than the work done at the leaf nodes $n^{\log_b a}$. Therefore, the leaves dictate the overall complexity.
  - *Solution:* $T(n) = \Theta\left(n^{\log_b a}\right)$
- **Case 2: Work is evenly distributed**
  - *Condition:* If $f(n) = \Theta\left(n^{\log_b a}\right)$.
  - *Interpretation:* The work done at the root is exactly the same order of magnitude as the work done at the leaves. The work is distributed evenly across all $\log n$ levels of the recursion tree.
  - *Solution:* We simply multiply by a logarithmic factor. $T(n) = \Theta\left(n^{\log_b a} \log n\right)$
- **Case 3: The root dominates the tree (Merging/Dividing is heavier)**
  - *Condition:* If $f(n) = \Omega\left(n^{\log_b a + \epsilon}\right)$ for some constant $\epsilon > 0$, AND it satisfies the regularity condition $(af(n/b) \leq cf(n)$ for some constant $c < 1)$.
  - *Interpretation:* The work done at the root $f(n)$ is polynomially overwhelmingly larger than the work done in the recursive calls. Therefore, the root completely dictates the overall complexity.
  - *Solution:* $T(n) = \Theta\left(f(n)\right)$

Q22. Apply a master method to give the tight asymptotic bounds of the following recurrences: (i) $T(n) = 4T(n/2) + n^2$ (ii) $T(n) = 9T(n/3) + n$.

Answer:

Solution for (i) $T(n) = 4T(n/2) + n^2$

- **Step 1: Extract the constants.**

  Comparing with $T(n) = aT(n/b) + f(n)$, we get: $a = 4$ $b = 2$ $f(n) = n^2$

- **Step 2: Calculate $n^{\log_b a}$.**

  $n^{\log_2 4}$ Since $2^2 = 4$, $\log_2 4 = 2$. Therefore, $n^{\log_b a} = \mathbf{n^2}$.

- **Step 3: Compare $f(n)$ and $n^{\log_b a}$.**

  Here, $f(n) = n^2$ and $n^{\log_b a} = n^2$. Since both are exactly the same, $f(n) = \Theta\left(n^{\log_2 4}\right)$.

- **Step 4: Apply the appropriate case.**

  This perfectly matches **Case 2** of the Master Method (Work is evenly distributed). *Solution Formula:* $T(n) = \Theta\left(n^{\log_b a} \log n\right)$ **Final Answer:** $T(n) = \Theta(n^2 \log n)$

Solution for (ii) $T(n) = 9T(n/3) + n$

- **Step 1: Extract the constants.**

  Comparing with $T(n) = aT(n/b) + f(n)$, we get: $a = 9$ $b = 3$ $f(n) = n$

- **Step 2: Calculate $n^{\log_b a}$.**

  $n^{\log_3 9}$ Since $3^2 = 9$, $\log_3 9 = 2$. Therefore, $n^{\log_b a} = \mathbf{n^2}$.

- **Step 3: Compare $f(n)$ and $n^{\log_b a}$.**

  Here, $f(n) = n^1$ and $n^{\log_b a} = n^2$. Since n^1 is polynomially strictly smaller than n^2 by a factor of n^1 (i.e., $n = O(n^{2-1})$), we have an $\epsilon = 1 > 0$.

- **Step 4: Apply the appropriate case.**

  Because $f(n)$ is polynomially smaller, this matches **Case 1** of the Master Method (The leaves dominate). *Solution Formula:* $T(n) = \Theta(n^{\log_b a})$ **Final Answer:** $T(n) = \Theta(n^2)$

Q23. Define the substitution method to solve a recurrence relation. Solve the following recurrence relation using substitution method: T(n) = 2T(n/2) + n.

Answer:

What is the Substitution Method?

The Substitution Method is a mathematical technique used to solve recurrence relations. It involves two main steps:

- **Step 1 (The Guess):** We make an educated guess for the asymptotic bound (the solution) of the recurrence relation.
- **Step 2 (The Proof):** We use the Principle of Mathematical Induction to prove that our guess is correct. We substitute the guessed solution back into the recurrence to show that it holds true for the constants chosen.

Solving $T(n) = 2T(n/2) + n$ using the Substitution Method:

- **The Guess:** By looking at the recurrence (which resembles the Merge Sort algorithm), we guess that the time complexity is $O(n\log\ n)$.

  Mathematically, this means we must prove that $T(n) \leq c \cdot n\log\ n$ for some positive constant c > 0 and sufficiently large n.

- **The Inductive Hypothesis:** Assume that our guess holds true for smaller values, specifically for n/2.

  Therefore, we assume:

$$T(n/2) \leq c \cdot (n/2)\log\,(n/2)$$

- **The Substitution Step:** Now, substitute this assumption back into the original recurrence relation:

$$T(n) = 2T(n/2) + n$$

$$T(n) \leq 2[c \cdot (\frac{n}{2})\log\,(\frac{n}{2})] + n$$

- **Simplifying the Equation:**

  Cancel out the 2s:

$$T(n) \leq c \cdot n\log\,(\frac{n}{2}) + n$$

Using the logarithm property $\log(a/b) = \log a - \log b$:

$$T(n) \leq c \cdot n(\log n - \log 2) + n$$

Assuming base 2 logarithms ($\log_2 2 = 1$):

$$T(n) \leq c \cdot n(\log n - 1) + n$$

$$T(n) \leq c \cdot n\log n - c \cdot n + n$$

- **Finding the Constant:**

  We want to prove that $T(n) \leq c \cdot n\log n$.

  For this to be true, the remaining residual terms must be less than or equal to zero:

  $$-c \cdot n + n \leq 0$$

  Divide by n:

  $$-c + 1 \leq 0 \implies c \geq 1$$

- **Conclusion:**

  Since we have found a valid constant ($c \geq 1$) that makes the mathematical inequality hold true, our initial guess is proven correct.

  Final Solution: $T(n) = O(n\log n)$

Q24. Define a recurrence relation of QuickSort algorithm and solve it using a recurrence tree.

Answer:

Recurrence Relation of QuickSort:

The time complexity of the QuickSort algorithm heavily depends on how the "pivot" element divides the array. Therefore, it has two primary recurrence relations:

- **Worst-Case Recurrence:** This occurs when the pivot is extremely unbalanced (e.g., the array is already sorted), dividing an array of size n into one subarray of size 0 and another of size n-1.

  $$T(n) = T(n-1) + O(n)$$

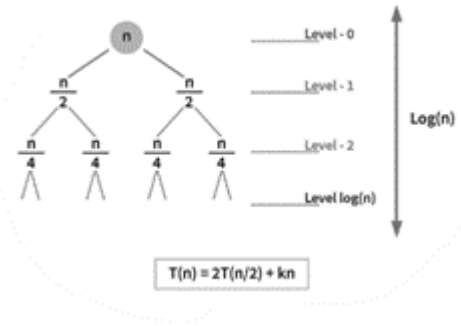
- **Best/Average-Case Recurrence:** This occurs when the pivot perfectly divides the array into two equal (or nearly equal) halves.

  $$T(n) = 2T(n/2) + c \cdot n$$

  (Where $c \cdot n$ represents the O(n) time taken by the partitioning process).

Solving the Best/Average-Case Recurrence using a Recurrence Tree:

Let us construct a recurrence tree for T(n) = 2T(n/2) + cn.



- **Level 0 (Root):** The cost of partitioning the full array is **cn**. The problem is divided into two subproblems of size n/2.
- **Level 1:** The cost of partitioning the two subproblems is c(n/2) + c(n/2) = **cn**. The problem is further divided into four subproblems of size n/4.
- **Level 2:** The cost is 4 × c(n/4) = **cn**.
- **Leaf Level:** The tree continues to branch out until the subproblem size reaches 1.

Mathematical Calculation from the Tree:

- **Cost per level:** As observed, the total cost at every single horizontal level of the tree sums up to exactly **cn**.
- **Height of the tree:** The subproblem size strictly halves at each level ($n, n/2, n/4, \ldots, 1$). The number of times you can divide n by 2 until you reach 1 is $\log_2 n$. Therefore, the height of the tree is $\log_2 n$.
- **Total Work Done:**

$$\text{Total Cost} = (\text{Cost per level}) \times (\text{Height of the tree})$$

$$\text{Total Cost} = cn \times \log_2 n$$

Final Solution: Discarding the constant c, the asymptotic time complexity derived from the tree is $O(n \log n)$.

Q25. Solve the given recurrence relation using recursion tree method: T(n) = 2T(n/2) + n.

Answer:

1. Understanding the Recurrence:

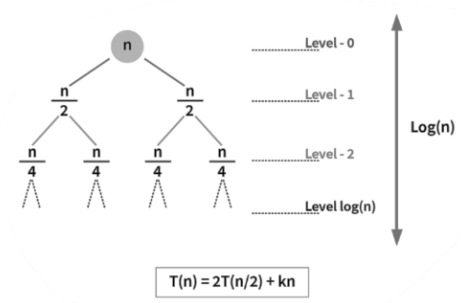
The given recurrence relation is T(n) = 2T(n/2) + n.

This tells us two things:

- **The Cost of Division/Merging:** The non-recursive part is n. This means at each step, the algorithm performs n operations to divide the problem or merge the results.
- **The Recursive Calls:** The algorithm divides the main problem of size n into **2 subproblems**, each of size **n/2**.

2. Building the Recursion Tree:

Let's construct the tree level by level to calculate the total work done.



- **Level 0 (Root):** The cost of the work done at the root is **n**.
- **Level 1:** The root splits into 2 subproblems of size n/2. The cost at this level is 2 × n/2) = **n**.
- **Level 2:** Those 2 subproblems split again, creating 4 subproblems of size n/4. The cost at this level is 4 × (n/4) = **n**.
- **Level 3:** 8 subproblems of size n/8. Cost is 8 × (n/8) = **n**.
- **Leaf Level:** This splitting continues until the subproblem size reduces to 1 (the base case).

Mathematical Calculation:

- **Cost per level:** As we can see from the tree, the sum of the work done at *every single level* is exactly **n**.
- **Height of the tree:** The problem size is repeatedly divided by 2 ($n \to n/2 \to n/4 \to \cdots \to 1$). The total number of divisions required to reach 1 is $\log_2 n$. Therefore, the height (or depth) of the tree is $\log_2 n$.
- **Total Work Done:** To find the total time complexity, we multiply the cost per level by the total number of levels (height).

$$\text{Total Cost} = (\text{Cost per level}) \times (\text{Height of the tree})$$

$$\text{Total Cost} = n \times \log_2 n$$

Final Solution: The tight asymptotic bound for this recurrence relation is $\Theta(n\log\ n)$. *(Note: This is the exact time complexity of the Merge Sort algorithm).*

Q26. Write and explain linear search algorithm. Mention best case, worst case and average case scenarios of linear search.

Answer:

What is the Linear Search Algorithm?

Linear Search (also known as Sequential Search) is the simplest searching algorithm. It works by sequentially checking each element in a list or array, starting from the very first element, until a match is found or the entire list has been searched. It does not require the array to be sorted.

Algorithm / Pseudocode:

Algorithm LinearSearch(Array, n, target)
// Array: The array to be searched
// n: The total number of elements in the Array
// target: The specific value we are looking for

Step 1: Set i = 0
Step 2: If i > n-1, go to Step 7
Step 3: If Array[i] == target, go to Step 6
Step 4: Set i = i + 1
Step 5: Go to Step 2
Step 6: Print "Element found at index i" and EXIT
Step 7: Print "Element not found" and EXIT

Explanation:

The algorithm uses a simple loop to iterate through the array. It compares the target value with Array[0], then Array[1], then Array[2], and so on. If it finds an exact match, it immediately stops and returns the index. If it reaches the end of the array without finding a match, it concludes that the target does not exist in the array.

Complexity Scenarios:

Scenario	Condition	Time Complexity	Explanation

Best Case	The target element is found at the very **first index** (index 0).	**O(1)**	The algorithm only makes 1 comparison and terminates immediately. It takes constant time.
Worst Case	The target element is at the **last index**, or it is **not present** in the array at all.	**O(n)**	The algorithm is forced to scan and compare every single element in the array of size n, making exactly n comparisons.
Average Case	The target element is found somewhere in the **middle** of the array.	**O(n)**	On average, the algorithm will have to search through half of the array (n/2 comparisons). Since we drop constants in asymptotic notation, O(n/2) simplifies to O(n).

Q27. Write and explain binary search algorithm with a suitable example. How many comparisons are needed for a binary search in a set of 512 elements?

Answer:

What is the Binary Search Algorithm?

Binary Search is a fast and efficient search algorithm that works strictly on **sorted arrays**. It follows the Divide and Conquer approach. Instead of scanning every element one by one (like linear search), it repeatedly divides the search interval in half. If the target value is less than the middle element, the algorithm narrows its search to the lower half. Otherwise, it searches the upper half.

Algorithm / Pseudocode:

Algorithm BinarySearch(Array, n, target)
// Array: A SORTED array
// n: Total number of elements
// target: The value being searched

Step 1: Set low = 0, high = n - 1
Step 2: While (low <= high), do:
Step 3: Calculate mid = (low + high) / 2
Step 4: If Array[mid] == target, return mid (Element Found!)
Step 5: If Array[mid] < target, set low = mid + 1 (Search right half)
Step 6: If Array[mid] > target, set high = mid - 1 (Search left half)
Step 7: End While
Step 8: Return -1 (Element Not Found)

Suitable Example:

Let's search for the target **23** in the sorted array: [2, 5, 8, 12, 16, 23, 38, 56, 72, 91] (Here, n = 10).

- **Iteration 1:** low = 0, high = 9.

  mid = (0 + 9) / 2 = 4.

  Array[4] = 16. Since 16 < 23, we discard the left half. We update low = mid + 1 = 5.

- **Iteration 2:** low = 5, high = 9.

  mid = (5 + 9) / 2 = 7.

  Array[7] = 56. Since 56 > 23, we discard the right half. We update high = mid - 1 = 6.

- **Iteration 3:** low = 5, high = 6.

mid = (5 + 6) / 2 = 5.

Array[5] = 23. This matches our target! The algorithm successfully returns index **5**.

Comparisons for 512 elements:

In the worst-case scenario, the maximum number of comparisons required for Binary Search is calculated using the formula: $\mathbf{[\log_2(n)] + 1.}$

For n = 512:

Maximum comparisons = $\log_2(512) + 1$

Since $2^9 = 512$, we know $\log_2(512) = 9$.

Maximum comparisons = 9 + 1 = **10**.

Conclusion: It will take a maximum of exactly **10 comparisons** to find any element (or conclude it is missing) in a sorted array of 512 elements.

Q28. Write recursive binary search algorithms and analyse its complexity in worst case scenario.

Answer:

Recursive Binary Search Algorithm:

A recursive implementation of Binary Search calls itself on the updated, smaller sub-arrays instead of using a while loop.

C/C++ Style Pseudocode:

```
// Initial call should be: RecursiveBinarySearch(Array, target, 0, n-1)

int RecursiveBinarySearch(int Array[], int target, int low, int high) {
   // Base Case: If the search space is exhausted, the element is not present
   if (low > high) {
      return -1;
   }

   // Find the middle index
   int mid = low + (high - low) / 2; // Prevents integer overflow

   // Case 1: Target is found at mid
   if (Array[mid] == target) {
      return mid;
   }
   // Case 2: Target is smaller than mid, search the left sub-array
   else if (target < Array[mid]) {
      return RecursiveBinarySearch(Array, target, low, mid - 1);
   }
   // Case 3: Target is larger than mid, search the right sub-array
   else {
      return RecursiveBinarySearch(Array, target, mid + 1, high);
   }
}
```

Complexity Analysis in the Worst Case Scenario:

- **Defining the Recurrence Relation:**

In the worst-case scenario (the target element is not in the array, or it is at the very ends of the array), the algorithm will repeatedly divide the array of size n into half (n/2) until the size becomes 1. At each step, comparing the target with Array[mid] takes constant time O(1).

Therefore, the recurrence relation is:

T(n) = T(n/2) + O(1)

- **Solving the Recurrence (Using Master Theorem):**

  Comparing T(n) = T(n/2) + 1 with the standard form T(n) = aT(n/b) + f(n):

  a = 1

  b = 2

  f(n) = 1 (or n^0)

  Now, we calculate $n^{\log_b a} = n^{\log_2 1} = n^0 = 1$.

  Since f(n) and $n^{\log_b a}$ are exactly equal (1 = 1), this falls strictly under **Case 2** of the Master Theorem.

  Formula for Case 2: $T(n) = \Theta(n^{\log_b a} \log\ n)$

  Substituting the values: $T(n) = \Theta(1 \cdot \log\ n)$

  Worst-Case Time Complexity: $O(\log\ n)$

- **Space Complexity Analysis (Worst Case):**

  Unlike the iterative version which uses O(1) space, the recursive version consumes stack memory for every function call. Since the recursion tree goes $\log_2 n$ levels deep, it creates $\log_2 n$ stack frames in memory.

  Worst-Case Space Complexity: $O(\log\ n)$

Q29. Explain Quick sort algorithm using divide and conquer approach. Apply the partition procedure of Quicksort algorithm to the following array: [35, 10, 40, 5, 60, 25, 45, 15].

Answer:

Quick Sort using Divide and Conquer: Quick Sort is a highly efficient sorting algorithm based on the Divide and Conquer strategy. It works in three major steps:

- **Divide (Partitioning):** The algorithm selects a specific element from the array called the **"Pivot"**. It then rearranges the array so that all elements smaller than the pivot are placed to its left, and all elements greater than the pivot are placed to its right. After this step, the pivot is exactly in its final sorted position.
- **Conquer:** The algorithm recursively applies the exact same process to the sub-array on the left of the pivot and the sub-array on the right of the pivot.
- **Combine:** Since the sub-arrays are sorted in place, no extra work is needed to combine them. The entire array is automatically sorted.

Applying the Partition Procedure: Let's apply the standard partitioning method to the given array: [35, 10, 40, 5, 60, 25, 45, 15]. *We will select the first element (35) as the Pivot.* We will use two pointers: a **Left pointer (L)** searching for elements greater than the pivot, and a **Right pointer (R)** searching for elements smaller than the pivot.

- **Initial Array:** [35(Pivot), 10, 40, 5, 60, 25, 45, 15]
  - L starts at 10 (index 1), R starts at 15 (index 7).
- **Step 1:** * L moves right until it finds an element > 35. L stops at 40.
  - R moves left until it finds an element < 35. R stops at 15.
  - **Swap** the elements at L and R (40 and 15).

- *Array becomes:* [35, 10, 15, 5, 60, 25, 45, 40]

- **Step 2:**
  - L continues right and finds 60 (> 35).
  - R continues left and finds 25 (< 35).
  - **Swap** the elements at L and R (60 and 25).
  - *Array becomes:* [35, 10, 15, 5, 25, 60, 45, 40]
- **Step 3:**
  - L continues right and finds 60 (> 35).
  - R continues left and finds 25 (< 35).
  - At this point, the pointers have crossed each other (L is now to the right of R). We stop the searching process.
- **Final Step (Place the Pivot):**
  - Swap the Pivot (35) with the element at the R pointer (25).
  - *Array becomes:* [25, 10, 15, 5, 35, 60, 45, 40]

Result: The partition procedure is complete. The pivot **35** is in its correct sorted position. The left sub-array [25, 10, 15, 5] contains all smaller elements, and the right sub-array [60, 45, 40] contains all larger elements.

Q30. Explain merge sort algorithm using divide and conquer approach. Also mention its best case and worst case time complexities. Apply the same to sort the array of elements: [7, 15, 5, 9, 20, 18, 25, 17, 11].

Answer:

Merge Sort using Divide and Conquer: Merge Sort is a stable, comparison-based sorting algorithm that strictly follows the Divide and Conquer paradigm.

- **Divide:** It repeatedly divides the unsorted array into two equal halves until each sub-array contains only one element (a single element is mathematically always sorted).
- **Conquer:** It recursively sorts the smaller sub-arrays.
- **Combine (Merge):** It takes two sorted sub-arrays and merges them together into a single, larger sorted array. It continues merging these arrays back up the tree until the entire array is rebuilt and fully sorted.

Time Complexities: Unlike Quick Sort, Merge Sort divides the array exactly in half every single time, regardless of how the data is arranged. Therefore, its performance is consistent.

- **Best Case Time Complexity:** $O(n\log n)$
- **Worst Case Time Complexity:** $O(n\log n)$
- *Space Complexity:* $O(n)$ because it requires a temporary auxiliary array to merge the elements.

Applying Merge Sort to [7, 15, 5, 9, 20, 18, 25, 17, 11]:

Phase 1: Divide (Splitting the array)

- Initial: [7, 15, 5, 9, 20, 18, 25, 17, 11]
- Split 1: [7, 15, 5, 9, 20] and [18, 25, 17, 11]
- Split 2: [7, 15, 5], [9, 20] and [18, 25], [17, 11]
- Split 3: [7, 15], [5], [9], [20] and [18], [25], [17], [11]
- Split 4: [7], [15], [5], [9], [20], [18], [25], [17], [11] *(All elements are now individual sorted units)*

Phase 2: Conquer and Combine (Merging the sub-arrays)

- Merge 1: [7, 15], [5], [9, 20], [18, 25], [11, 17]
- Merge 2: [5, 7, 15], [9, 20], [18, 25], [11, 17]
- Merge 3: [5, 7, 9, 15, 20] and [11, 17, 18, 25]
- Final Merge: Compare elements from both halves one by one and place them in order:

  5 < 11 -> [5] 7 < 11 -> [5, 7] 9 < 11 -> [5, 7, 9] 15 > 11 -> [5, 7, 9, 11] 15 < 17 -> [5, 7, 9, 11, 15] ...and so on...
- **Final Sorted Array: [5, 7, 9, 11, 15, 17, 18, 20, 25]**

Q31. Multiply the following two matrices using Strassen's algorithm.

Answer:

What is Strassen's Algorithm? Standard matrix multiplication of two $n \times n$ matrices takes $O(n^3)$ time. Strassen's algorithm uses the Divide and Conquer approach to reduce the number of recursive multiplications required from 8 down to 7. This mathematically reduces the time complexity to $O(n^{\log_2 7}) \approx O(n^{2.81})$, making it significantly faster for large matrices.

The 7 Formulas of Strassen's Algorithm: Given two 2×2 matrices A and B, we want to find $C = A \times B$.

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}, \quad C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

Strassen computes the following **7 Products (P_1 to P_7)**:

9. $P_1 = A_{11} \times (B_{12} - B_{22})$
10. $P_2 = (A_{11} + A_{12}) \times B_{22}$
11. $P_3 = (A_{21} + A_{22}) \times B_{11}$
12. $P_4 = A_{22} \times (B_{21} - B_{11})$
13. $P_5 = (A_{11} + A_{22}) \times (B_{11} + B_{22})$
14. $P_6 = (A_{12} - A_{22}) \times (B_{21} + B_{22})$
15. $P_7 = (A_{11} - A_{21}) \times (B_{11} + B_{12})$

Then, the final quadrants of Matrix C are calculated as:

- $C_{11} = P_5 + P_4 - P_2 + P_6$
- $C_{12} = P_1 + P_2$
- $C_{21} = P_3 + P_4$
- $C_{22} = P_5 + P_1 - P_3 - P_7$

Step-by-Step Example Application: Let $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ and $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$

- **Calculate the 7 Products:**
  - $P_1 = 1 \times (6 - 8) = 1 \times (-2) = -2$
  - $P_2 = (1 + 2) \times 8 = 3 \times 8 = 24$
  - $P_3 = (3 + 4) \times 5 = 7 \times 5 = 35$
  - $P_4 = 4 \times (7 - 5) = 4 \times 2 = 8$
  - $P_5 = (1 + 4) \times (5 + 8) = 5 \times 13 = 65$
  - $P_6 = (2 - 4) \times (7 + 8) = -2 \times 15 = -30$
  - $P_7 = (1 - 3) \times (5 + 6) = -2 \times 11 = -22$
- **Calculate Matrix C elements:**
  - $C_{11} = 65 + 8 - 24 + (-30) = 19$
  - $C_{12} = -2 + 24 = 22$
  - $C_{21} = 35 + 8 = 43$
  - $C_{22} = 65 + (-2) - 35 - (-22) = 63 - 35 + 22 = 50$
- **Final Result:** $C = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$

Q32. Multiply the following two numbers using Karatsuba's algorithm.

Answer:

What is Karatsuba's Algorithm? Standard multiplication of two n-digit numbers takes $O(n^2)$ time. Karatsuba's algorithm is a fast multiplication algorithm that uses the Divide and Conquer approach to reduce the multiplication of two n-digit numbers to at most 3 multiplications of $n/2$-digit numbers. This reduces the time complexity to $O(n^{\log_2 3}) \approx O(n^{1.585})$.

The Karatsuba Formula: To multiply two n-digit numbers, X and Y:

- **Step 1:** Split both numbers into two halves (Left and Right).

  $X = X_L \cdot 10^{n/2} + X_R \; Y = Y_L \cdot 10^{n/2} + Y_R$

- **Step 2:** Compute exactly 3 multiplications:

    1. $P_1 = X_L \times Y_L$
    2. $P_2 = X_R \times Y_R$
    3. $P_3 = (X_L + X_R) \times (Y_L + Y_R)$

- **Step 3:** Combine the results using the formula:

$$\text{Result} = P_1 \cdot 10^n + (P_3 - P_1 - P_2) \cdot 10^{n/2} + P_2$$

Step-by-Step Example Application: Let's multiply $X = 12$ and $Y = 34$. (Here, the total number of digits is $n = 2$, so $n/2 = 1$).

- **Step 1: Split the numbers:**
    - For $X = 12$: $X_L = 1, X_R = 2$
    - For $Y = 34$: $Y_L = 3, Y_R = 4$
- **Step 2: Calculate the 3 Products (P_1, P_2, P_3):**
    - $P_1 = X_L \times Y_L = 1 \times 3 = \mathbf{3}$
    - $P_2 = X_R \times Y_R = 2 \times 4 = \mathbf{8}$
    - $P_3 = (X_L + X_R) \times (Y_L + Y_R) = (1 + 2) \times (3 + 4) = 3 \times 7 = \mathbf{21}$
- **Step 3: Combine using the Formula:**

$$\text{Result} = P_1 \cdot 10^2 + (P_3 - P_1 - P_2) \cdot 10^1 + P_2$$

$$\text{Result} = 3 \cdot 100 + (21 - 3 - 8) \cdot 10 + 8$$

$$\text{Result} = 300 + (10) \cdot 10 + 8$$

$$\text{Result} = 300 + 100 + 8 = \mathbf{408}$$

(Verification: 12×34 *is indeed* 408*. The algorithm works perfectly!)*

Q33. Give a divide and conquer based algorithm to find the i^{th} smallest element in an array of size n: Trace your algorithm to find 3rd smallest in the array: A = [10, 2, 5, 15, 50, 6, 20, 25].

Answer:

Divide and Conquer Algorithm (QuickSelect):

The algorithm to find the i^{th} smallest element efficiently is called **QuickSelect**. It is heavily based on the QuickSort algorithm. Instead of sorting the entire array, it uses the "Partition" function to place a Pivot element in its correct sorted position. If the pivot's position matches the desired index i, we stop. If i is less than the pivot's position, we recursively search only the left sub-array. If i is greater, we search only the right sub-array.

Algorithm Pseudocode:

Algorithm QuickSelect(A, low, high, i)
Step 1: If low == high, return A[low]
Step 2: Pivot_Index = Partition(A, low, high) // Standard QuickSort partition
Step 3: Let k = Pivot_Index - low + 1 // k is the relative position of the pivot
Step 4: If i == k, return A[Pivot_Index] // Element found!
Step 5: If i < k, return QuickSelect(A, low, Pivot_Index - 1, i) // Search Left
Step 6: If i > k, return QuickSelect(A, Pivot_Index + 1, high, i - k) // Search Right

Tracing the Algorithm for the 3rd smallest element (i = 3):

Given Array: A = [10, 2, 5, 15, 50, 6, 20, 25]

We need the **3rd smallest**, so our target position is i = 3. We will use the last element as the pivot for partitioning.

- **Iteration 1 (Entire Array):** [10, 2, 5, 15, 50, 6, 20, 25]
    - **Pivot:** 25.

    - **Partitioning:** Elements < 25 go left, > 25 go right.
    - *Array becomes:* [10, 2, 5, 15, 6, 20, 25, 50]
    - **Pivot Position:** 25 is now at position **7** (1-based index).
    - **Check:** We need the 3rd element. Since 3 < 7, we discard the right side and recursively search the **Left sub-array**.
- **Iteration 2 (Left Sub-array):** [10, 2, 5, 15, 6, 20]
    - **Target:** Still looking for i = 3.
    - **Pivot:** 20 (Last element of current sub-array).
    - **Partitioning:** Elements < 20 go left.
    - *Array becomes:* [10, 2, 5, 15, 6, 20] (No change, as all elements are smaller than 20).
    - **Pivot Position:** 20 is at position **6**.
    - **Check:** Since 3 < 6, we search the **Left sub-array**.
- **Iteration 3 (Left Sub-array):** [10, 2, 5, 15, 6]
    - **Target:** Still looking for i = 3.
    - **Pivot:** 6.
    - **Partitioning:** Elements < 6 go left ([2, 5]), > 6 go right ([10, 15]).
    - *Array becomes:* [2, 5, 6, 15, 10]
    - **Pivot Position:** The pivot 6 is perfectly placed at position **3**.
    - **Check:** Target i = 3 exactly matches the pivot's current position!

Final Output: The algorithm terminates and returns **6**.

(Verification: The sorted array is [2, 5, 6, 10, 15, 20, 25, 50]. The 3rd element is indeed 6).

Q34. Write Bubble Sort Algorithm. Using bubble sort, sort the following sequence: 20, 15, 25, 10, 8, 38, 27, 11. Find the total number of comparisons.

Answer:

Bubble Sort Algorithm:

Bubble Sort is a simple comparison-based sorting algorithm. It repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. This process "bubbles" the largest remaining element to its correct position at the end of the array in every pass.

Pseudocode:

```
Algorithm BubbleSort(A, n)
Step 1: For i = 0 to n - 1 do:
Step 2:     For j = 0 to n - i - 2 do:
Step 3:         If A[j] > A[j+1] then:
Step 4:             Swap A[j] and A[j+1]
Step 5:     End For
Step 6: End For
```

Sorting the sequence: 20, 15, 25, 10, 8, 38, 27, 11 (n = 8)

- **Pass 1:** (Compares up to the last index)
    - (20, 15) → Swap → 15, 20
    - (20, 25) → No Swap
    - (25, 10) → Swap → 10, 25
    - (25, 8) → Swap → 8, 25
    - (25, 38) → No Swap
    - (38, 27) → Swap → 27, 38
    - (38, 11) → Swap → 11, 38
    - *Array after Pass 1:* [15, 20, 10, 8, 25, 27, 11, 38] *(38 is in correct position)*
- **Pass 2:** (Compares up to the 2nd last index)
    - Swaps occur for: (20, 10), (20, 8), (27, 11)
    - *Array after Pass 2:* [15, 10, 8, 20, 25, 11, 27, 38] *(27 is in correct position)*
- **Pass 3:**
    - Swaps occur for: (15, 10), (15, 8), (25, 11)

- *Array after Pass 3:* [10, 8, 15, 20, 11, 25, 27, 38] *(25 is in correct position)*
- **Pass 4:**
  - Swaps occur for: (10, 8), (20, 11)
  - *Array after Pass 4:* [8, 10, 15, 11, 20, 25, 27, 38] *(20 is in correct position)*
- **Pass 5:**
  - Swaps occur for: (15, 11)
  - *Array after Pass 5:* [8, 10, 11, 15, 20, 25, 27, 38] *(15 is in correct position)*
- **Pass 6 & 7:**
  - The array is already fully sorted. The algorithm makes comparisons but no swaps occur.
  - *Final Sorted Array:* **[8, 10, 11, 15, 20, 25, 27, 38]**

3. Finding the Total Number of Comparisons:

In standard Bubble Sort, the number of comparisons made does not depend on the data; it is strictly mathematical.

For n elements, the algorithm makes:

- Pass 1: n-1 comparisons
- Pass 2: n-2 comparisons
- ...
- Pass n-1: 1 comparison

Formula for total comparisons: $C = \frac{n(n-1)}{2}$

For our array, n = 8:

$$C = \frac{8 \times (8-1)}{2} = \frac{8 \times 7}{2} = \frac{56}{2} = 28$$

Total Number of Comparisons = 28.

Q35. Sort the following sequence of numbers, using selection sort. Also find the number of comparisons and copy operations required by the algorithm in sorting this list: 28, 13, 12, 28, 35, 11, 15, 9, 36.

Answer:

Selection Sort Algorithm Concept:

Selection sort divides the input list into two parts: a sorted sublist built up from left to right, and an unsorted sublist occupying the rest of the array. In each pass, it finds the absolute minimum element in the unsorted sublist and swaps it with the leftmost unsorted element, effectively expanding the sorted boundary by one.

Step-by-Step Sorting Trace (n = 9):

- **Initial Array:** [28, 13, 12, 28, 35, 11, 15, 9, 36]
- **Pass 1:** Find minimum from index 0 to 8. The minimum is **9**. Swap it with the element at index 0 (28).
  - *Array:* [9, 13, 12, 28, 35, 11, 15, 28, 36]
- **Pass 2:** Find minimum from index 1 to 8. The minimum is **11**. Swap it with index 1 (13).
  - *Array:* [9, 11, 12, 28, 35, 13, 15, 28, 36]
- **Pass 3:** Find minimum from index 2 to 8. The minimum is **12**. It is already at index 2, so swap it with itself (no physical change).
  - *Array:* [9, 11, 12, 28, 35, 13, 15, 28, 36]
- **Pass 4:** Find minimum from index 3 to 8. The minimum is **13**. Swap it with index 3 (28).

- *Array:* [9, 11, 12, 13, 35, 28, 15, 28, 36]

- **Pass 5:** Find minimum from index 4 to 8. The minimum is **15**. Swap it with index 4 (35).
  - *Array:* [9, 11, 12, 13, 15, 28, 35, 28, 36]
- **Pass 6:** Find minimum from index 5 to 8. The minimum is **28**. Swap it with index 5 (28).
  - *Array:* [9, 11, 12, 13, 15, 28, 35, 28, 36]
- **Pass 7:** Find minimum from index 6 to 8. The minimum is **28**. Swap it with index 6 (35).
  - *Array:* [9, 11, 12, 13, 15, 28, 28, 35, 36]
- **Pass 8:** Find minimum from index 7 to 8. The minimum is **35**. It is already at index 7.
  - *Array:* [9, 11, 12, 13, 15, 28, 28, 35, 36]

  (The final element 36 is automatically in its correct place. Sorting is complete.)

Number of Comparisons and Copy Operations:

- **Total Comparisons:** In Selection Sort, the number of comparisons is fixed regardless of the initial arrangement. It is calculated as n(n-1)/2.

  For n = 9: Total comparisons = $9 \times \frac{8}{2} = \frac{72}{2} = 36$ **comparisons**.

- **Total Copy Operations:** The algorithm performs exactly n-1 swaps. Each swap operation requires 3 data movements (or copy operations) via a temporary variable (e.g., temp = a; a = b; b = temp).

  Swaps = 9 - 1 = 8 swaps.

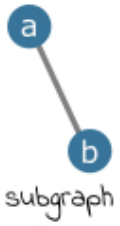
  Total copy operations = 8 × 3 = **24 copy operations**.

Q36. Explain any three of the following terms with the help of a suitable diagram: (i) Subgraph (ii) Connected graph (iii) Adjacency matrix (iv) Directed acyclic graph.

Answer:

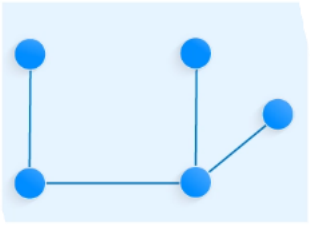
(i) Subgraph:

A subgraph is a graph formed by selecting a subset of vertices and a subset of edges from a larger, original graph. Formally, if G = (V, E) is a graph, then G' = (V', E') is a subgraph of G if all vertices in V' belong to V, and all edges in E' belong to E.



(ii) Connected Graph:

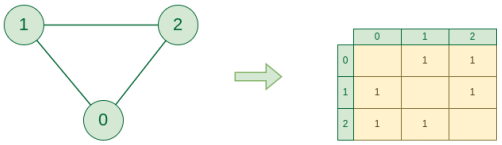
An undirected graph is considered a connected graph if there is at least one valid path between every possible pair of vertices. This means you can travel from any starting node to any other node in the graph without encountering a disconnected, unreachable island of nodes.



Connected Graph

(iii) Adjacency Matrix:

An adjacency matrix is a 2D array (matrix) used to represent a finite graph in computer memory. For a graph with V vertices, it is a V × V matrix. If there is an edge connecting vertex i to vertex j, the cell at row i and column j is marked with a **1** (or the edge weight). If there is no direct edge, the cell is marked with a **0**.

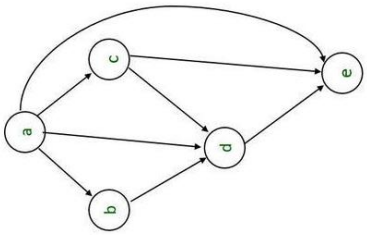


Undirected Graph Adjacency Matrix

Graph Representation of Undirected graph to Adjacency Matrix

(iv) Directed Acyclic Graph (DAG):

A Directed Acyclic Graph is a specialized directed graph that flows strictly in one direction and has absolutely **no cycles**. This means if you start traversing from a specific vertex following the direction of the arrows, it is impossible to ever loop back to your starting vertex. They are heavily used in task scheduling and dependency resolution.

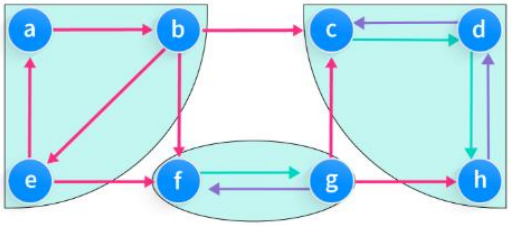


Q37. What are strongly connected components? Explain how adjacency matrix and adjacency list are created for a connected graph with the help of a suitable diagram.

Answer:

Strongly Connected Components (SCC):

In a **directed graph**, a Strongly Connected Component is a maximal subgraph in which there is a valid directed path from *every* vertex to *every other* vertex within that subgraph. In simple terms, if you pick any two vertices A and B inside an SCC, you can travel from A to B, and you can also travel back from B to A following the direction of the arrows.



Adjacency Matrix and Adjacency List Creation:

Let's take a simple connected undirected graph with 4 vertices (A, B, C, D) and 4 edges: (A-B), (A-C), (B-C), (C-D).

A) Adjacency Matrix:

An adjacency matrix is a 2D array of size V × V (where V is the number of vertices). If there is an edge between vertex i and vertex j, the cell at row i and column j is filled with 1. Otherwise, it is 0.

- For our graph, the matrix (4 × 4) looks like this:
  - Row A: [0, 1, 1, 0] (Edges to B and C)
  - Row B: [1, 0, 1, 0] (Edges to A and C)
  - Row C: [1, 1, 0, 1] (Edges to A, B, and D)
  - Row D: [0, 0, 1, 0] (Edge to C)

B) Adjacency List:

An adjacency list represents a graph as an array of linked lists. The size of the array is equal to the number of vertices. Each index of the array stores a list of all the adjacent vertices connected to it. It is much more memory-efficient than a matrix for sparse graphs.

- For our graph, the list looks like this:
  - A → [B] -> [C] -> NULL
  - B → [A] -> [C] -> NULL
  - C → [A] -> [B] -> [D] -> NULL
  - D → [C] -> NULL

Q38. Explain Depth First Search (DFS) algorithm with a suitable example. Apply DFS to the complete graph on four vertices. List the vertices in the order they would be visited.

Answer:

Depth First Search (DFS) Algorithm:

DFS is a standard graph traversal algorithm. Its strategy is to go as "deep" as possible down a single path before backtracking.

- It starts at a chosen root (or source) vertex.
- It marks the current vertex as **visited**.
- It picks the first unvisited adjacent neighbor and recursively applies DFS to it.
- If it reaches a vertex that has no unvisited neighbors (a dead end), it **backtracks** to the previous vertex and explores the next unvisited branch.
- It uses a **Stack** data structure (implicitly via recursion or explicitly).

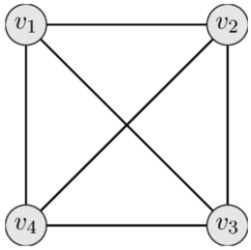
Applying DFS to a Complete Graph on 4 Vertices (K_4):

A complete graph on 4 vertices (let's name them 1, 2, 3, 4) means every single vertex is directly connected to every other vertex.

Edges = (1,2), (1,3), (1,4), (2,3), (2,4), (3,4).

Let's trace the DFS starting from vertex **1**:

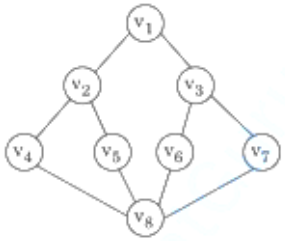
- **Step 1:** Start at **1**. Mark 1 as visited.
  - Neighbors of 1 are 2, 3, 4. We pick the smallest numerically: 2.
- **Step 2:** Go to **2**. Mark 2 as visited.
  - Neighbors of 2 are 1, 3, 4. 1 is already visited. Unvisited are 3, 4. We pick 3.
- **Step 3:** Go to **3**. Mark 3 as visited.
  - Neighbors of 3 are 1, 2, 4. 1 and 2 are already visited. Unvisited is 4. We pick 4.
- **Step 4:** Go to **4**. Mark 4 as visited.
  - Neighbors of 4 are 1, 2, 3. All are already visited!
- **Step 5 (Backtracking):** Vertex 4 backtracks to 3. 3 has no more unvisited neighbors, so it backtracks to 2. 2 has no more, backtracks to 1. The algorithm terminates.



Final Order of Visited Vertices:

The exact sequence in which the vertices were discovered is: **1 → 2 → 3 → 4**.

Q39. Apply the DFS algorithm to the following graph with the starting vertex v_1. List the order in which vertices will be visited. Show the complexity analysis if a graph is represented through (i) Adjacency list, and (ii) Adjacency matrix.



Answer:

Defining a Sample Graph:

Let us assume an undirected graph with 5 vertices: v_1, v_2, v_3, v_4, v_5.

Edges: $(v_1, v_2), (v_1, v_3), (v_2, v_4), (v_3, v_4), (v_4, v_5)$.

Applying DFS starting from v_1:

Depth First Search (DFS) dives as deep as possible into the graph before backtracking. We resolve ties by visiting the vertex with the smaller alphabetical/numerical index first.

- **Step 1:** Start at v_1. Mark it as visited. (Unvisited neighbors: v_2, v_3). Go to v_2.
- **Step 2:** Visit v_2. Mark as visited. (Unvisited neighbors: v_4). Go to v_4.
- **Step 3:** Visit v_4. Mark as visited. (Unvisited neighbors: v_3, v_5). Go to v_3.
- **Step 4:** Visit v_3. Mark as visited. (Neighbors are v_1, v_4, but both are already visited!). **Backtrack** to v_4.
- **Step 5:** Back at v_4, check remaining unvisited neighbors. Go to v_5.
- **Step 6:** Visit v_5. Mark as visited. No more neighbors. Backtrack all the way to the start. The algorithm terminates.

Order of Visited Vertices: $v_1 \rightarrow v_2 \rightarrow v_4 \rightarrow v_3 \rightarrow v_5$

Complexity Analysis:

The time complexity of DFS depends entirely on how the graph is stored in the computer's memory:

- **(i) If represented through an Adjacency List:** The algorithm visits every vertex exactly once, which takes O(V) time. For each vertex, it examines its adjacent edges. Across the entire traversal, every edge is examined exactly twice (once from each end in an undirected graph), taking O(E) time.
  - **Time Complexity: O(V + E)** *(where V is vertices and E is edges)*. This is highly efficient.
- **(ii) If represented through an Adjacency Matrix:**

  The algorithm visits every vertex once (V). However, to find the unvisited neighbors of a vertex, it must scan the entire row of size V in the matrix. Scanning V rows of size V results in V × V operations.

- **Time Complexity:** $O(V^2)$. This is slower and less efficient for sparse graphs.

Q40. Explain Breadth First Search (BFS) algorithm with a suitable example. Define a BFS tree. Give the breath first traversal for the undirected graph starting from vertex 'a'. Also give any three applications of DFS.

Answer:

Breadth First Search (BFS) Algorithm:

Unlike DFS, which dives deep, Breadth First Search explores the graph **level-by-level** (horizontally).

- It starts at a source vertex and visits all its immediate neighbors first.
- Only after all immediate neighbors are visited does it move on to the neighbors of those neighbors.
- It uses a **Queue (FIFO - First In, First Out)** data structure to keep track of the vertices that need to be explored next.

Suitable Example & Traversal starting from vertex 'a':

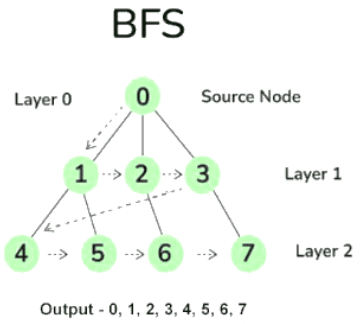
Assume a graph with vertices: a, b, c, d, e, f.

Edges: (a,b), (a,c), (b,d), (c,e), (d,f), (e,f).

- **Initialization:** Enqueue 'a'. Queue: [a]
- **Level 0:** Dequeue 'a'. Visit **a**. Enqueue unvisited neighbors 'b' and 'c'. Queue: [b, c]
- **Level 1:** * Dequeue 'b'. Visit **b**. Enqueue unvisited neighbor 'd'. Queue: [c, d]
  - Dequeue 'c'. Visit **c**. Enqueue unvisited neighbor 'e'. Queue: [d, e]
- **Level 2:**
  - Dequeue 'd'. Visit **d**. Enqueue unvisited neighbor 'f'. Queue: [e, f]
  - Dequeue 'e'. Visit **e**. Neighbor 'f' is already in the queue. Queue: [f]
- **Level 3:**
  - Dequeue 'f'. Visit **f**. No unvisited neighbors. Queue: []
- **Traversal Order:** $a \rightarrow b \rightarrow c \rightarrow d \rightarrow e \rightarrow f$

Define a BFS Tree:

A BFS Tree is a subgraph that is generated during the BFS traversal. It includes all the vertices of the original graph but only the edges that were actually used to discover new unvisited vertices (these are called "tree edges"). Edges that connect to already visited vertices are ignored (called "cross edges"). The BFS tree inherently represents the **shortest path** from the root node to any other node in an unweighted graph.



Three Applications of DFS (Depth First Search):

1. **Topological Sorting:** Used to schedule tasks or jobs where some tasks have prerequisites (represented as a Directed Acyclic Graph).
2. **Detecting Cycles in a Graph:** If during a DFS traversal, the algorithm encounters a "back-edge" (an edge pointing to an ancestor node that is currently in the recursion stack), it proves the graph has a cycle.
3. **Finding Strongly Connected Components:** DFS is the core engine behind Kosaraju's algorithm and Tarjan's algorithm, which are used to find SCCs in a directed graph.

Q41. Explain topological sorting with a suitable example. Can topological sorting be applied to a cyclic graph? Justify your answer.

Answer:

What is Topological Sorting?

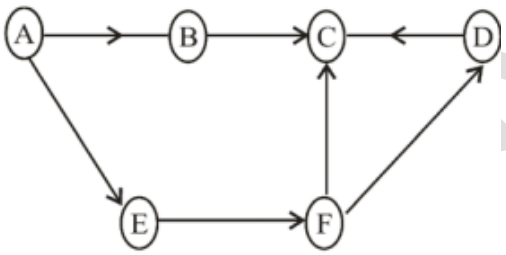
Topological sorting is a linear ordering of vertices in a **Directed Acyclic Graph (DAG)**. In this ordering, for every directed edge uv from vertex u to vertex v, vertex u must appear exactly before vertex v. It is widely used in scheduling tasks or jobs where certain tasks have prerequisites that must be completed first.

Suitable Example:

Imagine a student registering for university courses where some courses are prerequisites for others:

- **Course A:** Basic Programming (No prerequisite)
- **Course B:** Data Structures (Requires Course A)
- **Course C:** Web Development (Requires Course A)
- **Course D:** Advanced Algorithms (Requires Course B and Course C)

Graph Representation:



Applying Topological Sort (Kahn's Algorithm concept):

1. Find a vertex with no incoming edges (In-degree = 0). Here, it is **A**. Print **A** and remove it along with its outgoing edges.
    - *Sorted Order:* [A]
2. Now, **B** and **C** have no incoming edges. We can pick either. Let's pick **B**. Print **B** and remove its edges.
    - *Sorted Order:* [A, B]
3. Now, **C** has no incoming edges. Print **C** and remove its edges.
    - *Sorted Order:* [A, B, C]
4. Finally, **D** has no incoming edges. Print **D**.
    - *Final Sorted Order:* **[A, B, C, D]** *(Note: [A, C, B, D] is also a perfectly valid topological sort for this graph).*

Can it be applied to a cyclic graph?

No, topological sorting cannot be applied to any graph that contains a cycle.

Justification:

Topological sorting strictly requires that if there is an edge from u → v, then u must come before v.

If a graph has a cycle (e.g., $X \rightarrow Y \rightarrow Z \rightarrow X$), it creates a logical paradox:

- X must be completed before Y.
- Y must be completed before Z.
- Z must be completed before X.

This means X must happen before X, which is impossible to resolve into a linear sequence. Therefore, the graph must be acyclic (a DAG).

Q42. What are the similarities and differences between Dijkstra's algorithm and Prim's algorithm?

Answer:

Both Dijkstra's and Prim's algorithms are famous graph algorithms that operate very similarly under the hood, but they solve two completely different mathematical problems.

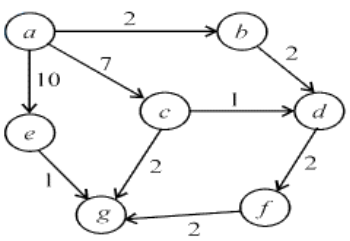
Similarities:

- **Algorithmic Paradigm:** Both algorithms use the **Greedy Approach**. They make the locally optimal choice at each step to find the global optimum.
- **Data Structures:** Both algorithms efficiently utilize a **Priority Queue** (often implemented as a Min-Heap) to constantly select the vertex with the minimum weight/cost.
- **Exploration Method:** Both maintain two sets of vertices during execution: a set of "visited" vertices and a set of "unvisited" vertices. They both grow a tree originating from a starting vertex, adding one node at a time.
- **Time Complexity:** When implemented using an adjacency list and a Min-Heap, both algorithms share the exact same time complexity: $O(E\log V)$, where E is edges and V is vertices.

Differences:

Feature	**Dijkstra's Algorithm**	**Prim's Algorithm**
Primary Goal	Finds the **Shortest Path** from a single source node to all other nodes in the graph.	Finds the **Minimum Spanning Tree (MST)** that connects all nodes with the absolute minimum total edge weight.
Graph Types	Works on both **Directed and Undirected** graphs (as long as edge weights are non-negative).	Strictly works on **Undirected, Connected** graphs.
Edge Selection Logic	Picks the edge that minimizes the **total accumulated distance from the source node**.	Picks the edge with the absolute minimum weight that connects **any node in the current tree** to an unvisited node.
Output	Produces a Shortest Path Tree, which may not necessarily contain the lightest edges of the whole graph.	Produces a Minimum Spanning Tree, which guarantees the lowest possible total edge weight.

Q43. Apply Dijkstra's algorithm to find the shortest path from a source vertex. Show the step-by-step trace.



Answer:

What is Dijkstra's Algorithm?

Dijkstra's algorithm is a Greedy algorithm used to find the shortest path from a single source vertex to all other vertices in a weighted graph (where all edge weights are non-negative).

Defining a Sample Graph:

Let us consider a graph with 5 vertices: **A, B, C, D, E**. Let the source vertex be **A**.

Edges and Weights:

- (A → B, weight 4)
- (A → C, weight 1)
- (C → B, weight 2)
- (B → D, weight 1)
- (C → D, weight 5)
- (D → E, weight 3)

Step-by-Step Trace:

We maintain a table with the shortest known distance to each vertex. Initially, the distance to the source (A) is 0, and all others are Infinity (∞). We also keep track of unvisited vertices.

Step	Unvisited Vertices	Chosen Vertex (Min Dist)	Distance to A	Distance to B	Distance to C	Distance to D	Distance to E
0	{A, B, C, D, E}	**A** (Dist 0)	**0**	∞	∞	∞	∞
1	{B, C, D, E}	**C** (Dist 1)	0	**4** (via A)	**1** (via A)	∞	∞
2	{B, D, E}	**B** (Dist 3)	0	**3** (via C)*	1	**6** (via C)	∞
3	{D, E}	**D** (Dist 4)	0	3	1	**4** (via B)*	∞
4	{E}	**E** (Dist 7)	0	3	1	4	**7** (via D)

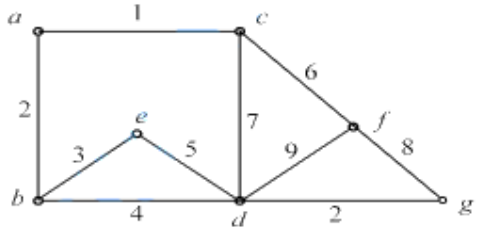
Explanation of Updates:

- *In Step 2:* From vertex C, we can reach B with a cost of 1 + 2 = 3. Since 3 is less than the previous known distance of 4, we update B's distance to 3.
- *In Step 3:* From vertex B, we can reach D with a cost of 3 + 1 = 4. Since 4 is less than the previous known distance of 6, we update D's distance to 4.

Final Shortest Paths from Source A:

- Path to A: Cost = 0
- Path to B: A → C → B (Cost = 3)
- Path to C: A → C (Cost = 1)
- Path to D: A → C → B → D (Cost = 4)
- Path to E: A → C → B → D → E (Cost = 7)

Q44. Apply Kruskal's or Prim's algorithm to find the Minimum Spanning Tree (MST) of a given graph.



Answer:

What is Kruskal's Algorithm?

Kruskal's algorithm is a Greedy algorithm that finds the Minimum Spanning Tree (MST) of an undirected, connected graph. It does this by sorting all the edges in increasing order of their weights and adding them to the MST one by one, strictly ensuring that no cycles are formed.

Defining a Sample Graph:

Let us consider a graph with 4 vertices: **1, 2, 3, 4**.

Edges and Weights:

- (1 - 3, weight 5)
- (3 - 4, weight 8)
- (1 - 2, weight 10)
- (2 - 3, weight 15)
- (1 - 4, weight 20)

3. Step-by-Step Trace (Kruskal's Algorithm):

Step 1: Sort all edges in ascending order of their weights.

1. Edge (1 - 3): Weight 5
2. Edge (3 - 4): Weight 8
3. Edge (1 - 2): Weight 10
4. Edge (2 - 3): Weight 15
5. Edge (1 - 4): Weight 20

Step 2: Pick edges one by one and check for cycles.

We need to add exactly V - 1 edges to complete the MST. Since we have 4 vertices, we need exactly 3 edges.

- **Pick Edge (1 - 3):** Weight 5.

  Does it form a cycle? No.

  Action: Add to MST. *(Edges in MST: 1)*

- **Pick Edge (3 - 4):** Weight 8.

  Does it form a cycle? No.

  Action: Add to MST. *(Edges in MST: 2)*

- **Pick Edge (1 - 2):** Weight 10.

  Does it form a cycle? No.

Action: Add to MST. *(Edges in MST: 3)*

We now have 3 edges in our MST. The algorithm terminates immediately.

4. Final Minimum Spanning Tree:

The edges included in the MST are: **(1 - 3), (3 - 4), and (1 - 2)**.

Total Minimum Cost: 5 + 8 + 10 = **23**.

Q45. What are the various tree traversal techniques? Explain with a suitable example.

Answer:

What is Tree Traversal?

Tree traversal is the algorithmic process of visiting (checking or updating) every single node in a tree data structure exactly once. Unlike linear data structures (like arrays or linked lists) which can only be traversed sequentially, trees can be traversed in multiple ways depending on the order in which the root and its subtrees are visited.

There are three primary **Depth-First Traversal** techniques for a Binary Tree:

A) Preorder Traversal (Root, Left, Right):

- **Algorithm:** 1. Visit the Root node first.

  2. Recursively traverse the Left subtree.

  3. Recursively traverse the Right subtree.

- **Usage:** Used to create a duplicate or copy of the tree.

B) Inorder Traversal (Left, Root, Right):

- **Algorithm:** 1. Recursively traverse the Left subtree.

  2. Visit the Root node.

  3. Recursively traverse the Right subtree.

- **Usage:** In a Binary Search Tree (BST), an inorder traversal will always yield the elements in strictly ascending (sorted) order.

C) Postorder Traversal (Left, Right, Root):

- **Algorithm:** 1. Recursively traverse the Left subtree.

  2. Recursively traverse the Right subtree.

  3. Visit the Root node last.

- **Usage:** Used to completely delete a tree from memory, because it guarantees that you delete the children before you delete the parent node.

Suitable Example Trace:

Let us construct a standard Binary Tree with nodes numbered 1 to 6:

- Root node is **1**.
- Node 1 has left child **2** and right child **3**.
- Node 2 has left child **4** and right child **5**.
- Node 3 has only a right child **6**.

Applying the traversal algorithms to this tree:

- **Preorder Sequence: 1, 2, 4, 5, 3, 6** *(Visits the parent before its children)*
- **Inorder Sequence: 4, 2, 5, 1, 3, 6** *(Visits the left side, then the middle, then the right side)*
- **Postorder Sequence: 4, 5, 2, 6, 3, 1** *(Visits the parent strictly after both of its children)*

Q46. Differentiate between Greedy technique and Divide and Conquer technique. Give examples of algorithms for each.

Answer:

Both "Greedy" and "Divide and Conquer" are fundamental algorithmic design paradigms, but they approach problem-solving in fundamentally different ways.

The Core Concepts:

- **Greedy Approach:** This technique builds up a solution piece by piece. At every single step, it makes the choice that looks best *at that exact moment* (the locally optimal choice), hoping that these local optimums will eventually add up to a globally optimal solution. It never looks back or reconsiders its past choices.
- **Divide and Conquer:** This technique takes a massive problem, breaks it down into completely independent subproblems of the same type, recursively solves those smaller subproblems, and then mathematically merges those small solutions together to form the final answer for the massive problem.

Key Differences Comparison:

Feature	**Greedy Approach**	**Divide & Conquer**
Problem Solving Strategy	Makes decisions step-by-step based on immediate local benefit.	Divides the problem into smaller, disjoint sub-problems, solves them, and merges them.
Optimization vs. Structure	Strictly used for **Optimization problems** (finding maximum profit, shortest path, etc.).	Primarily a structural approach used for **searching, sorting**, or breaking down complex math.
Decision Making	A decision is made *before* moving on. It never changes its mind later.	Decisions are made by resolving the deepest sub-problems *first*, then working upwards.
Optimal Solution Guarantee	**Does not** always guarantee a globally optimal solution (it can get trapped in local optimums).	**Always** guarantees the mathematically correct and optimal solution for the defined logic.
Speed & Efficiency	Generally faster and requires less memory, but less accurate for complex overlapping problems.	Takes more memory (due to recursive stack calls) but is highly reliable.

3. Standard Examples:

- **Greedy Algorithms:** * Dijkstra's Shortest Path Algorithm
    - Kruskal's & Prim's Minimum Spanning Tree Algorithms
    - Huffman Coding (Data Compression)
    - Fractional Knapsack Problem

- **Divide and Conquer Algorithms:**
  - Merge Sort
  - Quick Sort
  - Binary Search
  - Strassen's Matrix Multiplication
  - Karatsuba's Integer Multiplication

Q47. Write and explain the procedure to find a solution to maximum bipartite matching problem with the help of an example. Write and explain pseudocode for Ford-Fulkerson's algorithm.

Answer:

Maximum Bipartite Matching Problem:

A bipartite graph is a graph where the vertices can be divided into two disjoint sets, L (Left) and R (Right), such that every edge connects a vertex in L to one in R. A "matching" is a subset of edges where no two edges share the same vertex. The "Maximum Bipartite Matching" problem asks us to find a matching that contains the largest possible number of edges.

Procedure to solve using Max-Flow:

We can easily solve this by converting it into a Network Flow problem.

- **Step 1:** Create a "Super-Source" node (S) and connect it to all vertices in the Left set (L) with directed edges of capacity 1.
- **Step 2:** Direct all original bipartite edges from L to R and assign them a capacity of 1.
- **Step 3:** Create a "Super-Sink" node (T) and connect all vertices in the Right set (R) to it with directed edges of capacity 1.
- **Step 4:** Run the **Ford-Fulkerson Algorithm** from S to T. The maximum flow value will be exactly equal to the maximum bipartite matching.

Example:

Imagine 3 Applicants (A_1, A_2, A_3) and 3 Jobs (J_1, J_2, J_3).

- A_1 can do J_1 or J_2.
- A_2 can do J_1.
- A_3 can do J_2 *or* J_3.

We connect Source S to A_1, A_2, A_3. We connect J_1, J_2, J_3 to Sink T. All capacities are 1. The Max-Flow will push 1 unit through $S \to A_1 \to J_2 \to T, S \to A_2 \to J_1 \to T, and\ S \to A_3 \to J_3 \to T$. The maximum matching is 3.

Ford-Fulkerson Algorithm:

This algorithm computes the maximum flow in a flow network. It relies on finding "augmenting paths" (paths from source to sink where more flow can be pushed).

Pseudocode:

```
Algorithm FordFulkerson(Graph G, Node S, Node T):
  // Initialize flow in all edges to 0
  For each edge (u, v) in G:
    flow[u][v] = 0
    flow[v][u] = 0

  max_flow = 0

  // While there is an augmenting path from S to T in the residual graph
  While PathExists(S, T, residual_graph, path):
    // Find the bottleneck capacity (minimum edge capacity) along the path
    bottleneck = minimum_capacity_in(path)
```

```
    // Augment the flow along the path
    For each edge (u, v) in path:
        flow[u][v] = flow[u][v] + bottleneck   // Add flow to forward edge
        flow[v][u] = flow[v][u] - bottleneck   // Subtract flow from reverse edge

    // Add bottleneck to total maximum flow
    max_flow = max_flow + bottleneck

Return max_flow
```

Explanation of Pseudocode:

The algorithm starts with zero flow. It repeatedly searches for a valid path from the source (S) to the sink (T) that has available capacity (usually using Breadth-First Search or Depth-First Search). Once a path is found, it determines the "bottleneck"—the smallest available capacity on that path. It then pushes this bottleneck amount of flow through the path. Crucially, it updates the "residual graph" by adding forward flow and creating "reverse edges" (which allow the algorithm to mathematically undo a bad flow choice later). It stops when no more paths from S to T exist.

Q48. What is Greedy approach for problem solving? How does a Greedy algorithm work? Write the activities performed in Greedy method.

Answer:

What is the Greedy Approach?

The Greedy approach is a fundamental algorithmic paradigm used primarily for solving **optimization problems** (where you want to find the maximum or minimum value). It builds up a solution piece by piece, always choosing the next piece that offers the most immediate, obvious benefit.

How does a Greedy algorithm work?

A greedy algorithm works on the principle of **"Local Optimum."** At every single step of the problem, the algorithm makes the choice that looks best *right now*, without ever worrying about future consequences. It never goes back to reconsider its past choices. The hope is that by choosing a series of local optimums, the algorithm will eventually arrive at a "Global Optimum" (the overall best solution).

Note: Greedy algorithms are fast and simple, but they do not guarantee an optimal solution for every type of problem. They only work perfectly when the problem exhibits the "Greedy-Choice Property" and "Optimal Substructure."

Activities / Components Performed in the Greedy Method:

Every standard greedy algorithm is built using five key components or activities:

1. **Candidate Set:** A set of all available items, candidates, or choices from which the solution is created (e.g., all edges in a graph, all items for a knapsack).
2. **Selection Function:** The core greedy logic. This function is used to choose the absolute best candidate from the candidate set to be added to the solution next.
3. **Feasibility Function:** A check to determine if adding the newly selected candidate violates any rules or constraints of the problem (e.g., does adding this edge create a cycle? Does this item exceed the knapsack's weight limit?).
4. **Objective Function:** A mathematical formula that assigns a value to a solution or a partial solution. It measures how "good" the current solution is (e.g., total weight, total profit).
5. **Solution Function:** A final check that indicates when the algorithm has discovered a complete, valid solution and should terminate.

Q49. Differentiate between greedy approach and dynamic approach to solve an optimization problem.

Answer:

Both Greedy and Dynamic Programming (DP) approaches are powerful techniques used to solve optimization problems, but they have fundamentally different strategies for making decisions and guaranteeing the best result.

Core Philosophies:

- **Greedy Approach:** It follows the principle of "Local Optimum." At each step, it blindly makes the choice that looks the most profitable or optimal right at that specific moment. It never reconsiders or changes its past decisions.
- **Dynamic Programming Approach:** It follows the "Principle of Optimality" and exhaustively evaluates all possible choices. It breaks the problem down into overlapping subproblems, solves each subproblem once, saves the result (memoization or tabulation), and uses those saved results to build the guaranteed best "Global Optimum."

Detailed Differences:

Feature	Greedy Approach	Dynamic Programming (DP)
Decision Making	Makes a decision *before* solving the smaller subproblems.	Makes a decision only *after* solving all the relevant smaller subproblems.
Re-evaluation	Once a choice is made, it is permanent. It never looks back.	It constantly looks back at previously solved subproblems to make the best current choice.
Optimality Guarantee	**Does not** always guarantee a globally optimal solution (it can fail if the local best choice leads to a bad overall outcome).	**Always** guarantees the mathematically perfect, globally optimal solution.
Subproblem Dependency	Subproblems are entirely independent of each other.	Subproblems heavily overlap and depend on one another.
Efficiency (Time & Space)	Generally much faster and requires very little memory O(1) extra space.	Generally slower and requires significant memory to store the results of subproblems (table arrays).
Standard Examples	Fractional Knapsack, Dijkstra's Algorithm, Kruskal's Algorithm.	0/1 Knapsack, Floyd-Warshall Algorithm, Matrix Chain Multiplication.

Q50. What is a Minimum Cost Spanning Tree (MCST)? Write Generic MCST algorithm.

Answer:

What is a Minimum Cost Spanning Tree (MCST)?

To understand an MCST, we must first break down its components:

- **Spanning Tree:** For any connected, undirected graph with V vertices, a spanning tree is a subgraph that contains exactly all V vertices connected by exactly V - 1 edges, with absolutely **no cycles**.
- **Minimum Cost Spanning Tree (MCST):** In a weighted graph (where each edge has a numerical cost or distance), there can be many different valid spanning trees. The MCST (often just called MST) is the specific spanning tree whose total sum of edge weights is the absolute minimum possible compared to all other spanning trees.

The Generic MCST Algorithm:

All specific MCST algorithms (like Kruskal's and Prim's) are essentially variations of one fundamental "Generic" algorithm. The generic algorithm manages a set of edges, let's call it A. Initially, A is empty. Step by step, it adds a "safe edge" to A.

- **What is a Safe Edge?** A safe edge is an edge that can be added to the set A without creating a cycle, and it guarantees that the set A can still eventually become a valid Minimum Spanning Tree.

Generic Algorithm Pseudocode:

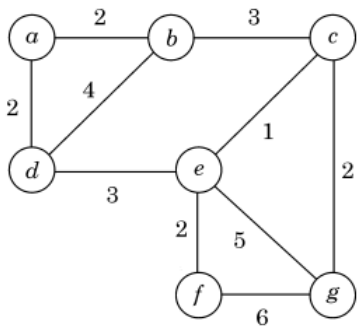
Algorithm Generic_MCST(Graph G, Weights W)
// G is a connected, undirected graph
// W represents the weights of the edges

Step 1: Initialize an empty set of edges: A = { }
Step 2: While the set A does not form a complete spanning tree (i.e., it has less than V - 1 edges), do:
Step 3: Find an edge (u, v) that is "safe" for set A.
Step 4: Add the safe edge to the set: A = A U {(u, v)}
Step 5: End While
Step 6: Return the set A (which is now the Minimum Cost Spanning Tree)

How specific algorithms implement Step 3:

- **Kruskal's Algorithm** finds the safe edge by always picking the lightest available edge in the entire graph that connects two disconnected trees (components).
- **Prim's Algorithm** finds the safe edge by always picking the lightest edge that connects a vertex currently *inside* the growing tree to a vertex *outside* the tree.

Q51. Write Prim's algorithm for finding Minimum Spanning Tree and find its time complexity. Also find MST of the following graph using Prim's algorithm.



Answer:

Prim's Algorithm: Prim's Algorithm is a greedy approach that builds the Minimum Spanning Tree (MST) one vertex at a time. It starts from an arbitrary root vertex and continuously adds the cheapest possible edge that connects a vertex already in the MST to a vertex that is not yet in the MST.

Pseudocode:

Algorithm Prim_MST(Graph, source):

Step 1: Initialize all vertices with a key value of Infinity.
Step 2: Set the key value of the source vertex to 0.
Step 3: Create a set (let's call it MST_Set) to keep track of vertices included in the MST. Initially, it is empty.
Step 4: While MST_Set does not contain all vertices:
Step 5: Pick the vertex 'u' which is not in MST_Set and has the minimum key value.
Step 6: Include 'u' in MST_Set.
Step 7: For every adjacent neighbor 'v' of 'u':
Step 8: If 'v' is not in MST_Set and the edge weight (u, v) is smaller than the current key of 'v':
Step 9: Update the key of 'v' to the edge weight (u, v).
Step 10: Set the parent of 'v' to 'u'.
Step 11: End While

Time Complexity:

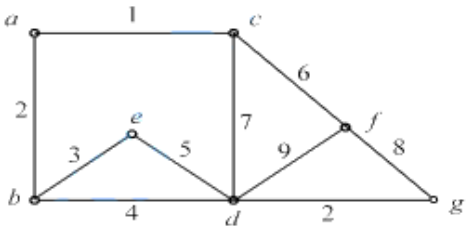
- If the graph is represented using an **Adjacency Matrix**, searching for the minimum key takes O(V) time, leading to an overall complexity of **O(V2)**.
- If the graph is represented using an **Adjacency List** and a **Min-Priority Queue** (Min-Heap), the time complexity is significantly reduced to **O(ElogV)**, where E is the number of edges and V is the number of vertices.

Finding MST using Prim's Algorithm (Example Trace): Let's assume a connected weighted graph with 4 vertices: **A, B, C, D**. Edges and Weights: (A-B, 2), (A-C, 3), (B-C, 1), (B-D, 4), (C-D, 5).*Let's start the algorithm from vertex **A**.*

Step	Visited Set (MST_Set)	Unvisited Vertices	Available Edges (from MST_Set to Unvisited)	Edge Chosen (Minimum Weight)
1	{A}	{B, C, D}	A-B (2), A-C (3)	**A-B (Weight 2)**
2	{A, B}	{C, D}	A-C (3), B-C (1), B-D (4)	**B-C (Weight 1)**
3	{A, B, C}	{D}	B-D (4), C-D (5)	**B-D (Weight 4)**
4	{A, B, C, D}	{ }	Algorithm Terminates.	None

Result: The edges included in the MST are **(A-B), (B-C), and (B-D)**.**Total Minimum Cost:** 2+1+4= **7**.

Q52. Write Kruskal's algorithm to find minimum cost spanning tree of the following graph. Show the step by step execution of Kruskal's algorithm.



Answer:

Kruskal's Algorithm: Kruskal's algorithm builds the Minimum Spanning Tree by treating the graph as a forest. It sorts all the edges in the graph in ascending order of their weights. It then picks the smallest edge and adds it to the MST, provided that adding this edge does not form a closed cycle. It repeats this until exactly V−1 edges are added.

Pseudocode:

Algorithm Kruskal_MST(Graph):
Step 1: Create an empty set 'MST_Edges' to store the final tree.
Step 2: Sort all the edges in the graph in non-decreasing (ascending) order of their weights.
Step 3: Initialize a Disjoint-Set (Union-Find) data structure to keep track of connected components.
Step 4: For each edge (u, v) from the sorted list:
Step 5: If the root of 'u' is not equal to the root of 'v' (i.e., adding the edge doesn't form a cycle):
Step 6: Add edge (u, v) to 'MST_Edges'.
Step 7: Perform Union(u, v) to merge their components.
Step 8: If 'MST_Edges' has exactly (V - 1) edges, stop the loop.
Step 9: Return 'MST_Edges'.

Step-by-Step Execution of Kruskal's Algorithm (Example Trace): Let's assume a graph with 5 vertices: **1, 2, 3, 4, 5**. We need exactly V−1=4 edges for our MST. Given Edges and Weights: (1-2, 1), (3-4, 2), (2-3, 3), (1-4, 4), (4-5, 5).

- **Step 1: Sort the edges by weight.**
    1. (1-2) : Weight 1
    2. (3-4) : Weight 2
    3. (2-3) : Weight 3
    4. (1-4) : Weight 4
    5. (4-5) : Weight 5
- **Step 2: Process edges one by one.**

Edge Considered	Weight	Does it form a cycle?	Action Taken	Current Edges in MST
(1-2)	1	No	**Accept**	{(1-2)}
(3-4)	2	No	**Accept**	{(1-2), (3-4)}
(2-3)	3	No	**Accept**	{(1-2), (3-4), (2-3)}
(1-4)	4	**Yes** (1-2-3-4-1)	**Reject**	{(1-2), (3-4), (2-3)}
(4-5)	5	No	**Accept**	{(1-2), (3-4), (2-3), (4-5)}

- **Step 3: Termination.** We have successfully collected 4 edges (V−1). The algorithm stops.

Result: The edges included in the MST are **(1-2), (3-4), (2-3), and (4-5)**.**Total Minimum Cost:** 1+2+3+5= **11**.

Q53. Define a fractional knapsack problem as an optimization problem. Write a greedy method to find an optimal solution to the problem. Show the complexity of the algorithm.

Answer:

Fractional Knapsack as an Optimization Problem:

The Fractional Knapsack problem is a classic optimization problem. We are given a set of n items, each having a specific weight w_i and a specific value or profit v_i. We also have a knapsack (a bag) with a maximum weight capacity W.

- **The Goal (Objective Function):** Maximize the total value of the items placed inside the knapsack.
- **The Constraint:** The total weight of the selected items must be less than or equal to the maximum capacity W.
- **The Fractional Aspect:** Unlike the 0/1 Knapsack problem where you must take an item entirely or leave it, here you are allowed to take **fractions** of an item (e.g., taking half an item gives you half its weight and half its value).

The Greedy Method / Algorithm:

The most optimal greedy strategy is to always pick the item that gives the maximum profit per unit of weight.

Pseudocode:

```
Algorithm FractionalKnapsack(Items, W, n)
// Items contain weight 'w' and value 'v' for 'n' items
// W is the maximum capacity of the knapsack

Step 1: For each item i from 1 to n:
Step 2:     Calculate the ratio R[i] = v[i] / w[i]
Step 3: Sort all items in DESCENDING order based on their ratio R.
Step 4: Initialize total_profit = 0, current_weight = 0

Step 5: For each item i in the sorted list:
Step 6:     If current_weight + w[i] <= W:
            // The whole item can fit
Step 7:         current_weight = current_weight + w[i]
Step 8:         total_profit = total_profit + v[i]
Step 9:     Else:
            // Only a fraction of the item can fit
Step 10:        remaining_capacity = W - current_weight
```

Step 11: fraction = remaining_capacity / w[i]
Step 12: total_profit = total_profit + (v[i] * fraction)
Step 13: Break the loop // Knapsack is completely full

Step 14: Return total_profit

Complexity of the Algorithm:

- Calculating the ratio for all items takes O(n) time.
- Sorting the items based on their ratio takes $O(n\log\ n)$ time using an efficient sorting algorithm like Merge Sort or Quick Sort.
- The final loop to fill the knapsack runs at most n times, taking O(n) time.
- **Total Time Complexity:** The sorting step dominates the execution time, so the overall time complexity is $O(n\log\ n)$.

Q54. What is Task Scheduling algorithm? Write pseudo code for task scheduling algorithm. Describe the task scheduling algorithm as an optimization problem and calculate its complexity. Apply the task scheduling algorithm to minimize the total amount of time spent in the system.

Answer:

What is the Task Scheduling Algorithm?

Task scheduling involves assigning a set of tasks to available resources (like a computer processor) over time. When the goal is to **minimize the total amount of time spent in the system** (which minimizes the average waiting time or completion time for all tasks), the most optimal Greedy approach is the **Shortest Processing Time First (SPT)** algorithm. It schedules the shortest tasks to run first.

Task Scheduling as an Optimization Problem:

Suppose we have n independent tasks, and each task i requires a specific execution time t_1.

- **The Goal (Objective Function):** Find a sequence or permutation of these tasks that minimizes the sum of their completion times.
- If we schedule tasks in the order $T_1, T_2, \dots, T_n$, the completion time of the first task is t_1. The second task finishes at $t_1 + t_2$. The total time spent by all tasks in the system is $(t_1) + (t_1 + t_2) + (t_1 + t_2 + t_3) + \cdots$
- To minimize this massive sum, the tasks with the smallest execution times must be placed at the beginning of the sequence.

Pseudocode for Shortest Processing Time First (SPT):

Algorithm MinimizeTotalTimeScheduling(Tasks, n)
// Tasks array contains the execution time 't' for 'n' tasks

Step 1: Sort the Tasks array in ASCENDING order based on execution time 't'.
// Now, t[1] <= t[2] <= ... <= t[n]

Step 2: Initialize total_time_spent = 0
Step 3: Initialize current_completion_time = 0

Step 4: For i = 1 to n:
Step 5: current_completion_time = current_completion_time + Tasks[i].t
Step 6: total_time_spent = total_time_spent + current_completion_time
Step 7: Print "Scheduled Task " + Tasks[i].id

Step 8: Return total_time_spent

Complexity of the Algorithm:

- The primary operation defining the efficiency of this algorithm is the sorting of the n tasks.
- Using an optimal comparison-based sorting algorithm, this takes $O(n\log\ n)$ time.

- Iterating through the sorted array to calculate the total time takes linear time, O(n).
- **Total Time Complexity:** $O(n\log\ n)$.

Application Example:

Assume we have 3 tasks: Task A (takes 5 mins), Task B (takes 2 mins), Task C (takes 8 mins).

- **Without Sorting (A, B, C):**
  - A finishes at 5.
  - B finishes at 5 + 2 = 7.
  - C finishes at 7 + 8 = 15.
  - Total time spent = 5 + 7 + 15 = **27**.
- **Applying SPT (Sorting ascending: B, A, C):**
  - B finishes at 2.
  - A finishes at 2 + 5 = 7.
  - C finishes at 7 + 8 = 15.
  - Total time spent = 2 + 7 + 15 = **24**.
- **Conclusion:** By applying the greedy algorithm, the total time spent in the system is successfully minimized.

Q55. What is Huffman coding? Write the steps for building the Huffman tree with an example. Construct an optimal Huffman tree and Huffman code for each character for the given set of frequencies.

Answer:

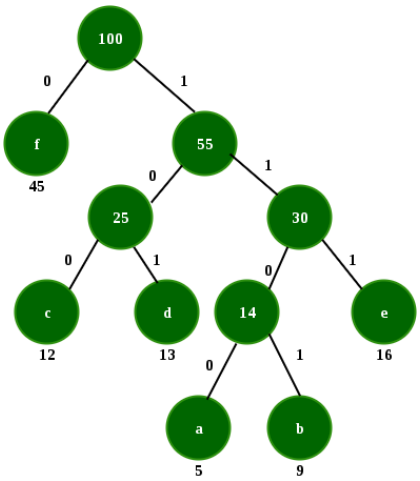
What is Huffman Coding? Huffman coding is a highly efficient Greedy algorithm used for **lossless data compression**. It reduces the size of data by assigning variable-length binary codes to characters based on their frequencies of occurrence. Characters that appear very frequently are assigned short codes (like 0 or 10), while extremely rare characters are assigned longer codes. This ensures the total number of bits required to store the message is minimized.

Steps for Building the Huffman Tree:

- **Step 1:** Create a leaf node for each unique character and build a Min-Priority Queue containing all these nodes, sorted by their frequencies in ascending order.
- **Step 2:** Extract the two nodes with the absolute lowest frequencies from the queue.
- **Step 3:** Create a new internal node. Its frequency will be the mathematical sum of the two extracted nodes' frequencies.
- **Step 4:** Make the first extracted node the left child (assign a bit value of 0) and the second extracted node the right child (assign a bit value of 1).
- **Step 5:** Insert this new internal node back into the Min-Priority Queue.
- **Step 6:** Repeat Steps 2 to 5 until there is only exactly one node left in the queue. This final node becomes the **Root** of the Huffman tree.

Example Construction & Huffman Codes: Let us assume the following characters and their frequencies: **A: 5, B: 9, C: 12, D: 13, E: 16, F: 45**

- **Trace:**
  - Initial Queue: {A:5, B:9, C:12, D:13, E:16, F:45}
  - *Merge 1:* Smallest are A(5) and B(9). Sum = 14 (Node N1). Queue becomes: {C:12, D:13, N1:14, E:16, F:45}
  - *Merge 2:* Smallest are C(12) and D(13). Sum = 25 (Node N2). Queue becomes: {N1:14, E:16, N2:25, F:45}
  - *Merge 3:* Smallest are N1(14) and E(16). Sum = 30 (Node N3). Queue becomes: {N2:25, N3:30, F:45}
  - *Merge 4:* Smallest are N2(25) and N3(30). Sum = 55 (Node N4). Queue becomes: {F:45, N4:55}
  - *Final Merge:* Smallest are F(45) and N4(55). Sum = 100 (Root).



- **Generating the Optimal Huffman Codes:** To find the code for a character, traverse from the Root down to the leaf node, appending 0 for a left turn and 1 for a right turn.
  - **F:** 0
  - **C:** 100
  - **D:** 101
  - **A:** 1100
  - **B:** 1101
  - **E:** 111

Q56. What is Dynamic Programming approach of problem solving? Write the steps involved in dynamic programming. Explain the principle of optimality in dynamic programming, with the help of an example.

Answer:

What is Dynamic Programming (DP)? Dynamic Programming is an algorithmic paradigm used to solve complex optimization problems by breaking them down into simpler, highly overlapping subproblems. Unlike Divide and Conquer (which solves independent subproblems), DP recognizes that subproblems overlap. To save time, it computes the solution to each subproblem exactly once and stores the result in a memory table (an array or matrix). When the same subproblem is encountered again, it simply looks up the stored answer instead of recomputing it.

Steps Involved in Dynamic Programming: Developing a DP algorithm generally follows four strict steps:

1. **Characterize the structure:** Define the mathematical structure of the optimal solution.
2. **Recursively define the value:** Create a mathematical recurrence relation that expresses the value of the optimal solution in terms of the values of its smaller subproblems.
3. **Compute the value (Bottom-Up):** Calculate the values of the optimal solutions for the subproblems in a bottom-up fashion, storing them in a table.
4. **Construct the final solution:** Use the computed information stored in the table to assemble the final optimal solution for the original problem.

The Principle of Optimality: Coined by Richard Bellman, the Principle of Optimality is the foundational rule that makes Dynamic Programming work. It states: *"An optimal solution to any instance of a problem always contains optimal solutions to all its subproblems."* If a problem does not strictly satisfy this principle, it **cannot** be solved using Dynamic Programming.

Example of the Principle of Optimality: Consider the **Shortest Path Problem**.

- Suppose you want to travel from City A to City C, and the absolutely shortest path goes directly through City B.
- The Principle of Optimality guarantees that the portion of your journey from City A to City B is the shortest possible path between A and B. Furthermore, the portion from City B to City C is the shortest possible path between B and C.
- If there was a shorter theoretical path from A to B, you would have taken it, which would make the overall path from A to C even shorter! Because the global path is optimal, the sub-paths *must* be optimal.

Q57. Explain the 0/1 Knapsack problem. Solve the 0/1 Knapsack problem using Dynamic Programming.

Answer:

What is the 0/1 Knapsack Problem? We are given a knapsack (a bag) with a maximum weight capacity W, and a set of n items. Each item has a specific weight wi and a specific value (profit) vi.

- **The Goal:** Maximize the total value of the items placed inside the knapsack without exceeding the capacity W.
- **The "0/1" Constraint:** Unlike the Fractional Knapsack, you cannot break items into pieces. You must either take the entire item (1) or leave it completely (0). Because of this strict 0/1 rule, the Greedy approach fails here, and we must use Dynamic Programming to guarantee an optimal solution.

Dynamic Programming Solution (The Recurrence Relation): We create a 2D table V[n+1][W+1], where rows represent items and columns represent weight capacities from 0 to W. The value at V[i][w] represents the maximum profit achievable using the first i items with a knapsack capacity of w.

- **Base Case:** V[i][0]=0 and V[0][w]=0 (0 profit if capacity is 0 or no items are chosen).
- **If the current item's weight is greater than the current capacity (wi>w):** We cannot include it. V[i][w]=V[i−1][w]
- **If the current item's weight fits in the capacity (wi≤w):** We take the maximum of two choices: *excluding* the item, or *including* the item (adding its value and subtracting its weight from the capacity). V[i][w]=max(V[i−1][w],V[i−1][w−wi]+vi)

Step-by-Step Example Solve: Let Knapsack Capacity **W=5**. Items (n=3):

- Item 1: Weight = 2, Value = 3
- Item 2: Weight = 3, Value = 4
- Item 3: Weight = 4, Value = 5

Constructing the DP Table V[i][w]:

i (Items) \ w (Capacity)	**0**	**1**	**2**	**3**	**4**	**5**
0 (No items)	0	0	0	0	0	0
1 (Item 1: w=2, v=3)	0	0	**3**	3	3	3
2 (Item 2: w=3, v=4)	0	0	3	**4**	4	**7**
3 (Item 3: w=4, v=5)	0	0	3	4	**5**	**7**

Trace of the last cell V[3][5] (Item 3, w=4, v=5; Capacity=5):

- Exclude Item 3: Profit = V[2][5] = 7
- Include Item 3: Profit = v3 + V[2][5 - 4] = 5 + V[2][1] = 5 + 0 = 5
- max(7,5)=7.

Result: The maximum profit that can be achieved is **7**. (By selecting Item 1 and Item 2, total weight = 5, total profit = 7).

Q58. What is Matrix chain multiplication problem? Explain how dynamic programming can be used to solve matrix chain multiplication. Find an optimal parenthesization of a matrix-chain product. List all the different orders in which we can multiply five matrices M1, M2, M3, M4, M5.

Answer:

What is the Matrix Chain Multiplication Problem? When multiplying a chain of matrices A1×A2×A3⋯×An, the final result is always the same regardless of how you group (parenthesize) them. However, the *number of scalar multiplications* required changes drastically based on the grouping.

- **The Goal:** Find the optimal way to place parentheses to **minimize** the total number of scalar multiplications. We are not actually multiplying the matrices; we are just finding the most efficient sequence to do so. *(Rule: Multiplying a matrix of size p×q with a matrix of size q×r takes exactly p×q×r multiplications).*

Solving with Dynamic Programming: We break the chain into two sub-chains at a splitting point k. Let m[i,j] be the minimum number of multiplications needed to compute the matrix Ai…Aj.

- **Base Case:** m[i,i]=0 (Cost of multiplying one matrix is 0).

- **Recurrence Relation:** For i<j,

  m[i,j]=i≤k<jmin{m[i,k]+m[k+1,j]+Pi−1PkPj}

  We build an m-table (to store minimum costs) and an s-table (to store the optimal split point k) in a bottom-up manner, calculating chains of length 2, then 3, and so on.

Example: Finding an Optimal Parenthesization: Let us have 3 matrices with dimension array P=[10,100,5,50]. This means: A1 (10×100), A2(100×5), A3(5×50).

- **Cost of (A1×A2)×A3:**
  - A1×A2 costs 10×100×5=5000. Result is a (10×5) matrix.
  - Result ×A3 costs 10×5×50=2500.
  - Total Cost = 5000+2500=7500.
- **Cost of A1×(A2×A3):**
  - A2×A3 costs 100×5×50=25000. Result is a (100×50) matrix.
  - A1× Result costs 10×100×50=50000.
  - Total Cost = 25000+50000=75000.
- **Conclusion:** The optimal parenthesization is **((A1×A2)×A3)** as it requires only 7,500 multiplications compared to 75,000!

Different orders to multiply Five Matrices (M1,M2,M3,M4,M5): The number of valid parenthesizations for n matrices is given by the Catalan number formula: Cn−1. For n=5, the number of ways is C4=51(48)=14 ways.

Here is the exhaustive list of all 14 different orders:

1. M1×(M2×(M3×(M4×M5)))
2. M1×(M2×((M3×M4)×M5))
3. M1×((M2×M3)×(M4×M5))
4. M1×((M2×(M3×M4))×M5)
5. M1×(((M2×M3)×M4)×M5)
6. (M1×M2)×(M3×(M4×M5))
7. (M1×M2)×((M3×M4)×M5)
8. (M1×(M2×M3))×(M4×M5)
9. ((M1×M2)×M3)×(M4×M5)
10. (M1×(M2×(M3×M4)))×M5
11. (M1×((M2×M3)×M4))×M5
12. ((M1×M2)×(M3×M4))×M5
13. ((M1×(M2×M3))×M4)×M5
14. (((M1×M2)×M3)×M4)×M5

Q59. What is all pair shortest path problem? Write and explain Floyd Warshall algorithm for shortest paths with the help of a diagram. Explain the working principle of Floyd-Warshall's algorithm. Apply Floyd Warshall's algorithm and show the matrix D^2 of the graph.

Answer:

What is the All-Pairs Shortest Path Problem?

Unlike Dijkstra's algorithm which finds the shortest path from a *single* starting node, the All-Pairs Shortest Path problem asks us to find the absolute shortest path distances between **every single pair of vertices** in a given edge-weighted directed graph.

Working Principle of Floyd-Warshall's Algorithm:

Floyd-Warshall uses the **Dynamic Programming** approach. Its core principle is to incrementally allow vertices to act as "intermediate" connecting points.

- It starts with a direct distance matrix (D^0), where it only considers direct edges between nodes.
- Then, it checks if routing the path through vertex 1 makes any existing paths shorter. If so, it updates the matrix to D^1.
- It then checks if routing through vertex 2 makes paths shorter, updating to D^2, and so on up to vertex k.
- The recurrence relation is:

$$D^k[i][j] = \min\left(D^{k-1}[i][j], D^{k-1}[i][k] + D^{k-1}[k][j]\right)$$

(In simple terms: "Is the direct distance from i to j smaller, or is the distance from i to k plus k to j smaller?")

Applying the Algorithm to Generate Matrix D^2:

Let us define a sample directed graph with 3 vertices: **1, 2, and 3**.

- Direct Edges: (1 → 2,weight8), (1 → 3,weight5), (2 → 1,weight3), (3 → 2,weight2).
- If there is no direct edge, the distance is ∞ (Infinity). Distance from a node to itself is 0.

Step 1: Initial Matrix D^0 (Direct Edges Only)

| | 1 | 2 | 3 |

| :--- | :--- | :--- | :--- |

| **1** | 0 | 8 | 5 |

| **2** | 3 | 0 | ∞ |

| **3** | ∞ | 2 | 0 |

Step 2: Matrix D^1 (Allowing vertex 1 as an intermediate node)

We check if any path can be shortened by traveling through node 1.

- We check path 2 → 3: Is (2 → 1) + (1 → 3) shorter than ∞?

  3 + 5 = 8. Since $8 < \infty$, we update $D[2][3]$ to 8.

- We check path 3 → 2: Is (3 → 1) + (1 → 2) shorter than 2?

  $\infty + 8 = \infty$. No update.

Matrix D^1:

| | 1 | 2 | 3 |

| :--- | :--- | :--- | :--- |

| **1** | 0 | 8 | 5 |

| **2** | 3 | 0 | **8** |

| **3** | ∞ | 2 | 0 |

Step 3: Matrix D^2 (Allowing vertex 2 as an intermediate node)

We check if any path can be shortened by traveling through node 2 (using values from D^1).

- We check path 1 → 3: Is (1 → 2) + (2 → 3) shorter than 5?

  8 + 8 = 16. Since 16 is not less than 5, no update.

- We check path 3 → 1: Is (3 → 2) + (2 \→ 1) shorter than ∞?

  2 + 3 = 5. Since 5 < ∞, we update D[3][1] to 5.

Matrix D^2 (Final Answer for the prompt):

| | 1 | 2 | 3 |

| :--- | :--- | :--- | :--- |

| **1** | 0 | 8 | 5 |

| **2** | 3 | 0 | 8 |

| **3** | **5** | 2 | 0 |

Q60. A binomial coefficient is defined by a recurrence relation. Write a recursive function to generate C(n, k).

Answer:

What is a Binomial Coefficient?

The binomial coefficient C(n, k) or $(\frac{n}{k})$ mathematically represents the number of ways to choose k items from a set of n distinct items without regarding the order.

The Recurrence Relation:

The binomial coefficient can be defined recursively using Pascal's Triangle property:

$$C(n,k) = C(n-1,k-1) + C(n-1,k)$$

Base Cases:

- C(n, 0) = 1 (There is exactly 1 way to choose 0 items from n items: choose nothing).
- C(n, n) = 1 (There is exactly 1 way to choose n items from n items: choose all of them).

Recursive Function in C/C++:

C

```
#include <stdio.h>

// Recursive function to generate Binomial Coefficient C(n, k)
int binomialCoeff(int n, int k) {
   // Base Cases
   if (k == 0 || k == n) {
      return 1;
```

```
    }

    // Recursive Step based on the recurrence relation
    return binomialCoeff(n - 1, k - 1) + binomialCoeff(n - 1, k);
}

int main() {
    int n = 5, k = 2;
    printf("Value of C(%d, %d) is %d\n", n, k, binomialCoeff(n, k));
    return 0;
}
```

Brief Analysis:

While this direct recursive implementation perfectly satisfies the definition, it is highly inefficient for large numbers because it recalculates the same overlapping subproblems multiple times (its time complexity is exponential, $O(2^n)$. In real-world applications, this is usually optimized using Dynamic Programming (building a 2D array to store the Pascal's triangle values) to bring the time complexity down to O(n × k).

Q61. What is string matching problem? What is string matching algorithm? Derive its best case time complexity.

Answer:

What is the String Matching Problem?

The string matching problem is a fundamental computational problem in computer science. Given a large body of text (called the **Text**, denoted as T with length n) and a smaller string you are looking for (called the **Pattern**, denoted as P with length m, where m \le n), the goal is to find all starting indices (or "valid shifts") in the text where the pattern occurs exactly.

What is a String Matching Algorithm?

A string matching algorithm is a step-by-step mathematical procedure designed to solve the string matching problem efficiently. These algorithms scan the text and compare it against the pattern. Popular examples include the Naïve String Matcher, the Knuth-Morris-Pratt (KMP) algorithm, and the Rabin-Karp algorithm. They are heavily used in search engines, text editors (Ctrl+F), and DNA sequence analysis.

Best Case Time Complexity (Standard Naïve Approach):

- **The Scenario:** The best-case scenario for standard string matching occurs when the very first character of the pattern does not match the character in the text at the current shift, causing the algorithm to immediately abort the inner loop and slide to the next position.
    - *Example:* Text T = "BBBBBBBB", Pattern P = "A".
- **Derivation:** * The total number of possible starting positions (shifts) for the pattern in the text is mathematically defined as (n - m + 1).
    - In the best case, the algorithm makes exactly **1** comparison at each of these positions before failing and moving on.
    - Total comparisons = 1 × (n - m + 1).
    - Therefore, the Best Case Time Complexity is **O(n - m + 1)**, which simplifies to **O(n)** (Linear Time) assuming the text is much larger than the pattern ($n \gg m$).

Q62. Explain the naïve string matching algorithm and derive its worst case complexity. What is its drawback? What will be the maximum valid shifts of a pattern in the text in the following example?

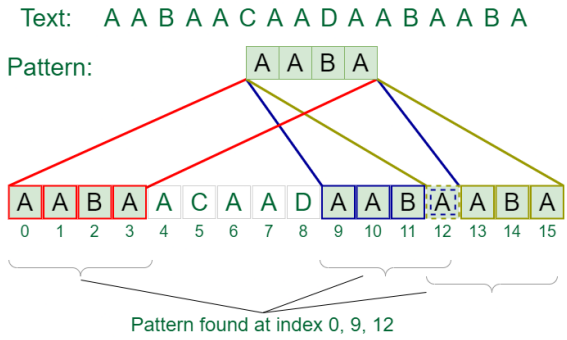
Answer:

The Naïve String Matching Algorithm:

The naïve (or brute-force) string matching algorithm is the simplest method. It works using a "sliding window" approach:

- It aligns the Pattern P with the beginning of the Text T (shift s = 0).

- It checks character by character from left to right.
- If all characters match, it records a "valid shift".
- If a mismatch occurs at any point, it stops checking, slides the pattern exactly **one position** to the right (shift s = s + 1), and starts comparing from the beginning of the pattern again.



Deriving the Worst Case Complexity:

- **The Scenario:** The worst-case occurs when all characters of the pattern match the text *except* for the very last character of the pattern. The algorithm is forced to make the maximum number of comparisons before failing.
  - *Example:* Text T = "AAAAAAAAAA", Pattern P = "AAAB".
- **Derivation:**
  - Number of possible shifts = (n - m + 1).
  - At every single shift, the algorithm makes exactly m comparisons before finding the mismatch at the end.
  - Total comparisons = m × (n - m + 1).
  - Therefore, the Worst-Case Time Complexity is strictly **O(mn)** (Quadratic Time).

What is its Drawback?

The primary drawback of the Naïve algorithm is that it is highly inefficient because it **ignores previously gathered information**. When a mismatch occurs, it completely forgets about the characters it just successfully matched and strictly shifts the pattern by only 1 position. This leads to massive amounts of redundant, repetitive comparisons.

4. Example: Maximum Valid Shifts

Let us define a standard example to trace:

- **Text T:** "AABAACAADAABAABA" (Length n = 16)
- **Pattern P:** "AABA" (Length m = 4)

We need to find the "valid shifts" (the starting indices where the pattern fully matches):

- Shift s = 0: $T[0 \ldots 3] =$ "AABA". **(Match 1)**
- Shift $s = 1 \ldots 8$: Mismatches occur.
- Shift s = 9: $T[9 \ldots 12] =$ "AABA". **(Match 2)**
- Shift s = 10, 11: Mismatches occur.
- Shift s = 12: $T[12 \ldots 15] =$ "AABA". **(Match 3)**

Result: The pattern occurs at shifts **0, 9, and 12**. The maximum number of valid shifts in this example is **3**. *(Note: The maximum number of total possible shifts tested by the algorithm is 16 - 4 + 1 = 13 shifts).*

Q63. Explain Knuth Morris Pratt (KMP) algorithm of string matching with a suitable example. Explain the process of building LPS array for a pattern 'P'.

Answer:

What is the Knuth-Morris-Pratt (KMP) Algorithm?

The KMP algorithm is an advanced string matching algorithm that significantly improves upon the Naïve method. Its core innovation is that it **never backtracks the text pointer**. When a mismatch occurs, KMP uses pre-processed information about the pattern itself to determine exactly how far it can safely shift the pattern forward without missing any potential matches. It does this using an auxiliary array called the **LPS array**.

What is the LPS Array?

LPS stands for **Longest Proper Prefix which is also a Suffix**.

- A **Proper Prefix** of a string is a prefix that is not equal to the string itself (e.g., proper prefixes of "ABC" are "A", "AB").
- A **Suffix** is the ending part of the string (e.g., suffixes of "ABC" are "C", "BC", "ABC").
- For a pattern P of length m, the LPS array is of size m. The value at LPS[i] stores the length of the longest proper prefix of the sub-pattern P[0...i] that exactly matches a suffix of the same sub-pattern.

Process of Building the LPS Array (With Example):

Let's build the LPS array for the Pattern **P = "ABABCABAB"** (Length m = 9).

We use two pointers: len (length of the previous longest prefix suffix, initialized to 0) and i (current index, starting at 1). LPS[0] is always 0.

- **i = 0:** Sub-pattern "A". LPS = 0. (Array: [0])
- **i = 1:** Sub-pattern "AB". Prefix "A" ≠ Suffix "B". LPS = 0. (Array: [0, 0])
- **i = 2:** Sub-pattern "ABA". Prefix "**A**" == Suffix "**A**". len becomes 1. LPS = 1. (Array: [0, 0, 1])
- **i = 3:** Sub-pattern "ABAB". Prefix "**AB**" == Suffix "**AB**". len becomes 2. LPS = 2. (Array: [0, 0, 1, 2])
- **i = 4:** Sub-pattern "ABABC". Mismatch between 'A' (index len) and 'C'. We backtrack len using previous LPS values until we find a match or len becomes 0. Here, it drops to 0. LPS = 0. (Array: [0, 0, 1, 2, 0])
- **i = 5:** Sub-pattern "ABABCA". Prefix "**A**" == Suffix "**A**". len becomes 1. LPS = 1. (Array: [0, 0, 1, 2, 0, 1])
- **i = 6:** Sub-pattern "ABABCAB". Prefix "**AB**" == Suffix "**AB**". len becomes 2. LPS = 2. (Array: [0, 0, 1, 2, 0, 1, 2])
- **i = 7:** Sub-pattern "ABABCABA". Prefix "**ABA**" == Suffix "**ABA**". len becomes 3. LPS = 3. (Array: [0, 0, 1, 2, 0, 1, 2, 3])
- **i = 8:** Sub-pattern "ABABCABAB". Prefix "**ABAB**" == Suffix "**ABAB**". len becomes 4. LPS = 4. (Array: [0, 0, 1, 2, 0, 1, 2, 3, 4])

Final LPS Array: [0, 0, 1, 2, 0, 1, 2, 3, 4]

Q64. Describe the basic principle of KMP algorithm for string matching. What is its advantage compared to Naive and Rabin-Karp's algorithm for string matching? Calculate the time complexity of KMP algorithm.

Answer:

Basic Principle of the KMP Algorithm:

Once the LPS array is built, the KMP algorithm starts scanning the Text T with a pointer i and the Pattern P with a pointer j.

- As long as T[i] == P[j], both pointers move forward (i++, j++).
- **The Magic of KMP:** If a mismatch occurs at P[j] after some characters have already matched, KMP *knows* that the characters prior to P[j] matched perfectly. It looks at the LPS array at the previous index (LPS[j-1]). This value tells the algorithm exactly how many characters of the pattern can safely bypass the comparison because they are guaranteed to match.
- We update the pattern pointer $j = \text{LPS}[j-1]$ and continue comparing with the *same* text pointer i. The text pointer i never goes backwards!

Advantages over Naïve and Rabin-Karp:

Feature	KMP vs. Naïve Algorithm	KMP vs. Rabin-Karp Algorithm

Backtracking	Naïve resets the text pointer backward upon every mismatch. KMP **never** moves the text pointer backward, making it much faster.	Both avoid naive text pointer backtracking, but their mechanisms differ.
Worst-Case Performance	Naïve's worst case is terrible: O(mn). KMP strictly guarantees a worst-case of **O(m + n)**.	Rabin-Karp's worst-case is also O(mn) if "Hash Collisions" occur frequently (spurious hits). KMP does not use hashing, so it is immune to collisions and guarantees O(m + n).
Best Use Case	Naïve is only good for tiny texts. KMP is excellent when the pattern has many repeating sub-patterns (e.g., DNA sequences).	Rabin-Karp is better for searching *multiple* different patterns simultaneously. KMP is vastly superior for single-pattern worst-case scenarios.

Time Complexity Calculation of KMP:

The KMP algorithm is divided into two distinct phases:

1. **Pre-processing Phase (Building the LPS Array):** The algorithm iterates through the pattern of length m exactly once to build the array. This takes **O(m)** time.
2. **Searching Phase (Scanning the Text):** The algorithm iterates through the text of length n. Even though there is a while loop inside the matching loop to update the j pointer, the text pointer i only ever increments and never decrements. Therefore, the text is scanned exactly once, taking **O(n)** time.

- **Total Time Complexity:** O(m) + O(n) = **O(m + n)** (Strictly Linear Time).
- **Space Complexity: O(m)** (Required to store the LPS array of size equal to the pattern length).

Q65. Explain the concept of rolling hash function applied in Rabin-Karp algorithm for string matching problem with the help of an example.

Answer:

What is the Rabin-Karp Algorithm?

The Rabin-Karp algorithm is a string matching algorithm that uses **hashing** to find a pattern P of length m within a text T of length n. Instead of comparing the pattern to the text character by character (like the Naïve algorithm), it computes a numerical "hash value" for the pattern and compares it to the hash value of each m-length window of the text. If the hash values match, it then checks the characters one by one to avoid "spurious hits" (hash collisions).

The Concept of the Rolling Hash Function:

The true power of Rabin-Karp lies in the **Rolling Hash Function**.

If we calculate the hash of every m-length window in the text from scratch, it would take O(m) time per window, leading to an inefficient O(mn) total time.

A "Rolling Hash" allows us to compute the hash of the *next* window in strictly **O(1) constant time** by reusing the hash value of the *previous* window. It does this by:

- **Dropping** the value of the first character of the old window.
- **Shifting** the remaining characters.
- **Adding** the value of the new character entering the window.

Example of a Rolling Hash Calculation:

Let us use a simple base-10 numerical example to understand the math easily.

- **Text T:** 23590
- **Pattern P:** 35 (Length m = 2)
- Let our hash function simply be the integer value of the window modulo 13 (a prime number to avoid collisions).

Step 1: Calculate Hash of Pattern:

- Pattern = 35 → Hash = $35 (mod 13) = 9$.

Step 2: Calculate Hash of First Text Window:

- Window 1 = 23 → Hash = $23 (mod 13) = 10$.
- *Check:* $10 \neq 9$. No match.

Step 3: Apply Rolling Hash for Window 2:

Instead of calculating 35 % 13 from scratch, we use the rolling formula:

New_Hash = (Old_Window - Leading_Digit * 10^(m-1)) * 10 + New_Digit

- Old Window = 23
- Leading Digit to drop = 2 (Since m=2, $10^{m-1} = 10^1 = 10$)
- New Digit to add = 5
- Next Window Integer = $(23 - 2 \times 10) \times 10 + 5 = (3) \times 10 + 5 = 35$.
- New_Hash = $35 (mod 13) = 9$.
- *Check:* 9 == 9. The hashes match! We perform a character-by-character check, confirm it is a true match, and successfully record the shift.

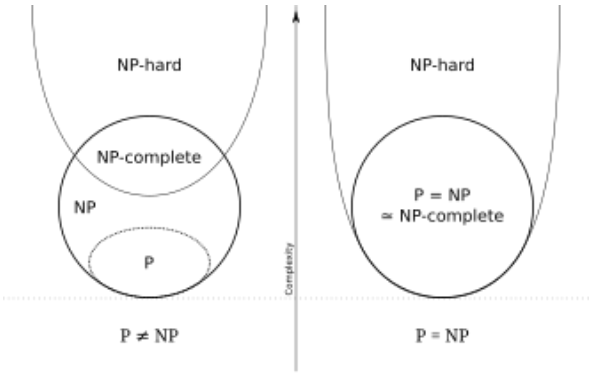
Q66. Explain P, NP and NP-complete class of problems with appropriate examples of each class. Differentiate between NP-Hard and NP-Complete problems.

Answer:

The Complexity Classes (P, NP, NP-Complete):

In computational complexity theory, decision problems (problems with a "Yes" or "No" answer) are categorized based on how fast they can be solved and how fast their solutions can be verified.

- **P Class (Polynomial Time):** * **Definition:** The set of all decision problems that can be strictly **solved** in polynomial time (O(n^k)) by a standard, Deterministic Turing Machine (a regular computer). These are considered "easy" or "tractable" problems.
    - **Examples:** Sorting a list, searching in an array, finding the shortest path (Dijkstra's Algorithm).
- **NP Class (Non-Deterministic Polynomial Time):**
    - **Definition:** The set of all decision problems where, if you are given a proposed solution (a "certificate"), you can **verify** whether that solution is correct or not in polynomial time using a standard computer. Alternatively, it is the set of problems that can be *solved* in polynomial time by a theoretical Non-Deterministic computer (one that can guess the perfect path every time).
    - **Note:** All problems in P are automatically in NP (if you can solve it quickly, you can definitely verify it quickly).
    - **Examples:** Sudoku (hard to solve, but if someone gives you a filled grid, checking if it is correct takes seconds), Traveling Salesperson Problem (decision version).
- **NP-Complete Class:**
    - **Definition:** These are the absolute **hardest problems within the NP class**. A problem is NP-Complete if:
        1. It belongs to the NP class (its solution can be verified quickly).
        2. Every single other problem in NP can be mathematically reduced (transformed) into this problem in polynomial time.
    - **Significance:** If anyone ever finds a fast, polynomial-time algorithm to solve just *one* NP-Complete problem, it would prove that P = NP (the biggest unsolved question in computer science).
    - **Examples:** Boolean Satisfiability Problem (SAT), Clique Problem, Vertex Cover Problem.



Differentiating Between NP-Hard and NP-Complete:

This is a very common point of confusion. Here is the strict difference:

Feature	**NP-Complete**	**NP-Hard**
Is it in NP?	**Yes.** Its solution *must* be verifiable in polynomial time.	**Not necessarily.** An NP-Hard problem does not even have to be in NP. You might not be able to verify its solution quickly.
Hardness Level	The hardest problems *inside* the NP boundary.	At least as hard as the hardest problems in NP, but can be infinitely harder.
Reduction	All NP problems can be reduced to it.	All NP problems can be reduced to it.
Examples	SAT Problem, Subset Sum Problem.	The Halting Problem (It is impossible to solve and impossible to verify quickly, so it is NP-Hard but NOT NP-Complete).

Q67. Explain the concept of non-deterministic algorithm with the help of an example. List the problems which belong to non-deterministic class of complexity.

Answer:

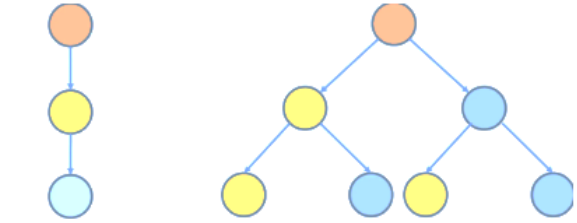
Concept of a Non-Deterministic Algorithm:

A non-deterministic algorithm is a theoretical model of computation. Unlike standard (deterministic) algorithms that follow exactly one predictable path of execution, a non-deterministic algorithm can explore multiple paths simultaneously, or conceptually, it has the magical ability to always "guess" the correct path if one exists.

It consists of two distinct phases:

- **Non-Deterministic Phase (Guessing Phase):** The algorithm magically guesses a proposed solution (a "certificate") from the vast space of all possible solutions.
- **Deterministic Phase (Verification Phase):** The algorithm takes this guessed solution and checks if it is correct using standard, step-by-step logic in polynomial time.

If the verification returns "True", the algorithm succeeds. The class **NP** (Non-Deterministic Polynomial time) is exactly defined by problems that can be solved by these theoretical non-deterministic algorithms in polynomial time.



Example of a Non-Deterministic Algorithm:

Let's look at the **Searching Problem** (finding an element x in an array A of n elements).

- *Standard (Deterministic) Approach:* Loop through the array from index 0 to n-1, checking each element one by one. Time taken: O(n).
- *Non-Deterministic Approach:*
    1. **Guess:** Magically guess an index i between 0 and n-1.
    2. **Verify:** Check if A[i] == x. If yes, print "Found".

    Time taken: O(1). The algorithm "knows" exactly where to look.

Problems belonging to the Non-Deterministic Class of Complexity (NP Class):

Virtually all decision problems whose solutions can be verified quickly belong here. Some famous hard problems in this class include:

- Boolean Satisfiability Problem (SAT)
- Traveling Salesperson Problem (Decision Version)
- Subset Sum Problem
- Graph Coloring Problem
- Hamiltonian Cycle Problem
- Knapsack Problem (Decision Version)

Q68. Explain Cook-Levin's theorem on CNF-Satisfiability problem, with the help of an example.

Answer:

What is Cook-Levin's Theorem?

The Cook-Levin theorem (formulated independently by Stephen Cook and Leonid Levin in the 1970s) is one of the most important theorems in computer science.

It states that the **Boolean Satisfiability Problem (SAT)** is **NP-Complete**.

This was a massive breakthrough because it was the very first problem ever proven to be NP-Complete. It means two things:

1. SAT is in the NP class (its solutions can be verified quickly).
2. Every single other problem in the NP class can be mathematically translated (reduced) into a SAT problem in polynomial time.

    Significance: If you can find a fast algorithm to solve SAT, you instantly have a fast algorithm to solve every other NP problem in existence!

What is the CNF-Satisfiability Problem?

CNF stands for **Conjunctive Normal Form**. A Boolean logic formula is in CNF if it is an AND of ORs.

- **Literal:** A variable (like x) or its negation (like $\neg x$).
- **Clause:** A group of literals connected by OR ($\vee$).

- **Formula:** A group of clauses connected by AND (∧).

The CNF-SAT problem asks: *"Is there any combination of TRUE or FALSE assignments to the variables that makes the entire formula evaluate to TRUE?"*

Example of CNF-SAT:

Let us take a simple CNF formula with two variables, x_1 and x_2:

$$F = (x_1 \vee x_2) \wedge (\neg x_1 \vee x_2)$$

- **Clause 1:** $(x_1 \vee x_2)$ means at least one of them must be TRUE.
- **Clause 2:** $(\neg x_1 \vee x_2)$ means either x_1 must be FALSE, or x_2 must be TRUE.

Let's try to satisfy it:

- *Guess 1:* $x_1 =$ TRUE, $x_2 =$ FALSE.
  - Clause 1: (TRUE ∨ FALSE) → TRUE.
  - Clause 2: (FALSE ∨ FALSE) → FALSE.
  - Overall: TRUE ∧ FALSE → FALSE. (Not satisfied)
- *Guess 2:* $x_1 =$ FALSE, $x_2 =$ TRUE.
  - Clause 1: (FALSE ∨ TRUE) → TRUE.
  - Clause 2: (TRUE ∨ TRUE) → TRUE.
  - Overall: TRUE ∧ TRUE → TRUE. (Satisfied!)

Because we found an assignment that makes the formula TRUE, we say this formula is **Satisfiable**. If the formula was extremely large, finding this combination deterministically would take exponential time, which is why SAT is so notoriously difficult to solve, yet so easy to verify once you have the guess!

Q69. Design a state space tree for the given subset sum problem. S = 4, 6, 7, 8, W = 8.

Answer:

What is the Subset Sum Problem?

The Subset Sum problem asks us to find if there is a subset of the given set S whose elements sum up exactly to a target weight **W**. We solve this using **Backtracking** by building a State Space Tree.

Designing the State Space Tree:

- **Given:** Set S = 4, 6, 7, 8, Target W = 8.
- **Logic:** At each level of the tree, we consider one element from the set. We have two choices:
  - **Left Branch:** Include the element in our subset (add its value to our current sum).
  - **Right Branch:** Exclude the element from our subset (current sum remains the same).
- If the sum exceeds W (8), we kill that branch (Dead End). If the sum exactly equals W (8), we have found a successful subset.

Step-by-Step Tree Trace:

- **Level 0 (Start):** Current Sum = 0.
- **Level 1 (Consider 4):**
  - Include 4: Sum = 4
  - Exclude 4: Sum = 0
- **Level 2 (Consider 6):**
  - From Sum=4 → Include 6: Sum = 10 (**Dead End**, 10 > 8)
  - From Sum=4 → Exclude 6: Sum = 4
  - From Sum=0 → Include 6: Sum = 6
  - From Sum=0 → Exclude 6: Sum = 0
- **Level 3 (Consider 7):**
  - From Sum=4 → Include 7: Sum = 11 (**Dead End**, 11 > 8)

- From Sum=4 → Exclude 7: Sum = 4
- From Sum=6 → Include 7: Sum = 13 (**Dead End**, 13 > 8)
- From Sum=6 → Exclude 7: Sum = 6
- From Sum=0 → Include 7: Sum = 7
- From Sum=0 → Exclude 7: Sum = 0

- **Level 4 (Consider 8):**
  - From Sum=4 → Include 8: Sum = 12 (**Dead End**)
  - From Sum=4 → Exclude 8: Sum = 4 (Fails, $4 \neq 8$)
  - From Sum=6 → Include 8: Sum = 14 (**Dead End**)
  - From Sum=6 → Exclude 8: Sum = 6 (Fails, $6 \neq 8$)
  - From Sum=7 → Include 8: Sum = 15 (**Dead End**)
  - From Sum=7 → Exclude 8: Sum = 7 (Fails, $7 \neq 8$)
  - From Sum=0 → Include 8: Sum = **8** (**SUCCESS!**)
  - From Sum=0 → Exclude 8: Sum = 0 (Fails)

Result: The algorithm successfully finds the subset **{8}** that exactly matches the target weight W = 8.

Q70. What are the key features of combinatorial problems? Describe and formulate three combinatorial problems.

Answer:

Key Features of Combinatorial Problems:

Combinatorial problems deal with finding an optimal object or counting specific arrangements from a finite set of discrete mathematical structures (like graphs, sets, or combinations).

- **Discrete State Space:** The solutions consist of discrete elements (integers, graphs) rather than continuous values (like real numbers).
- **Exponential Growth:** The number of possible solutions usually grows exponentially as the input size increases (2^n or n!), making brute-force searching impossible for large inputs.
- **Optimization Goal:** They typically ask to maximize or minimize a specific objective function (e.g., shortest path, maximum profit).

Three Combinatorial Problems:

- **A. Traveling Salesperson Problem (TSP):**
  - **Description:** A salesperson must visit n cities exactly once and return to the starting city. The goal is to find the tour with the absolute minimum total travel distance or cost.
  - **Formulation:** Given a complete weighted graph G = (V, E), find a Hamiltonian Cycle (a closed loop visiting every vertex exactly once) with the minimum sum of edge weights.
- **B. 0/1 Knapsack Problem:**
  - **Description:** Given a bag with a maximum weight capacity and a set of items (each with a weight and a value), determine which items to pack to maximize the total value without tearing the bag. You cannot take fractions of an item.
  - **Formulation:** Maximize $\sum(v_i \cdot x_i)$ subject to the constraint $\sum(w_i \cdot x_i) \leq W$, where $x_i \in \{0,1\}$.
- **C. N-Queens Problem:**
  - **Description:** Place N chess queens on an N × N chessboard such that no two queens can attack each other. (No two queens can share the same row, column, or diagonal).
  - **Formulation:** Find an arrangement of coordinates (r_i, c_i) for $1 \leq i \leq N$ such that for any two queens i and j, $r_i \neq r_j$, $c_i \neq c_j$, and $|\, r_i - r_j \,| \neq |\, c_i - c_j \,|$.

Q71. Short notes on:

Answer:

- **Deterministic vs. Non-deterministic Algorithms:**

  A **Deterministic Algorithm** follows a single, predictable sequence of steps. For a given input, it will always execute the exact same instructions and produce the same output in the same amount of time. A **Non-Deterministic Algorithm** is a theoretical model that can evaluate multiple execution paths simultaneously or "magically" guess the correct path to the

solution instantly. Deterministic machines represent real-world computers, while non-deterministic machines represent the theoretical limits of computation.

- **CLIQUE and Vertex Cover Problem:**

  Both are famous NP-Complete problems involving graphs.

  - A **CLIQUE** is a complete subgraph where every single vertex is directly connected to every other vertex in that subgraph. The Clique problem asks: "Does there exist a clique of size k in this graph?"
  - A **Vertex Cover** is a set of vertices such that every single edge in the graph touches at least one of the vertices in this set. The Vertex Cover problem asks: "Does there exist a vertex cover of size k?"
- **Backtracking Problem with Example:**

  Backtracking is an algorithmic technique for solving problems recursively by trying to build a solution incrementally, one piece at a time. If a piece violates the rules, the algorithm abandons it ("backtracks" to the previous step) and tries the next available option.

  - *Example:* **Sudoku**. You place a number 1-9 in an empty cell. If it breaks a rule (same number in the row/column), you backtrack, erase it, and try the next number.
- **Tractable vs. Intractable Problems:**
  - **Tractable Problems** are those that can be solved reasonably fast by a computer. Mathematically, they can be solved in Polynomial Time (O(n^k)). They belong to the P class (e.g., Sorting, Matrix Multiplication).
  - **Intractable Problems** are those that take an unacceptably long time to solve (Exponential Time, O(2^n) or O(n!)). For large inputs, even the world's fastest supercomputers would take millions of years to solve them (e.g., TSP, cracking complex cryptography).
- **Approximation Algorithms:**

  Since we cannot find exact, perfect solutions to NP-Hard optimization problems (like TSP) in a reasonable amount of time, we use Approximation Algorithms. These algorithms run fast (in polynomial time) and are mathematically guaranteed to find a solution that is "close enough" or "near-optimal" to the actual perfect solution.

Q72. Explain the three cases of the Master Theorem and solve the recurrence relation (e.g., T(n) = 9T(n/3) + n).

Answer:

Explanation of Master's Theorem:

The Master Theorem provides a direct formula to find the time complexity of recurrence relations that follow the Divide and Conquer strategy. It applies to recurrences of the form:

T(n) = aT(n/b) + f(n)

Where:

- n is the size of the problem.
- $a \geq 1$ is the number of subproblems.
- b > 1 is the factor by which the subproblem size is reduced.
- f(n) is the work done outside the recursive calls (dividing/merging cost).

To find the solution, we always compare f(n) with the term $n^{\log_b a}$.

The Three Cases:

- **Case 1 (Leaves Dominate):** If f(n) is polynomially *smaller* than $n^{\log_b a}$. (specifically, f(n) = $f(n) = O(n^{\log_b a - \epsilon})$ for some $\epsilon > 0$).
  - **Solution:** $T(n) = \Theta(n^{\log_b a})$
- **Case 2 (Work is Evenly Distributed):** If f(n) is exactly the *same* as $n^{\log_b a}$. (specifically, $f(n) = \Theta(n^{\log_b a})$)).
  - **Solution:** $T(n) = \Theta(n^{\log_b a} \log\ n)$
- **Case 3 (Root Dominates):** If f(n) is polynomially *larger* than $n^{\log_b a}$. (specifically, $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some $\epsilon > 0$) AND it satisfies the regularity condition.

- **Solution:** $T(n) = \Theta(f(n))$

2. Solving the Recurrence Relation: T(n) = 9T(n/3) + n

- **Step 1: Extract constants.**

  Comparing with T(n) = aT(n/b) + f(n):

  a = 9

  b = 3

  f(n) = n

- **Step 2: Calculate $n^{\log_b a}$.**

  $$n^{\log_3 9}$$

  Since $3^2 = 9$, $\log_3 9 = 2$.

  So, $n^{\log_b a} = n^2$.

- **Step 3: Compare f(n) and $n^{\log_b a}$.**

  Here, $f(n) = n^1$ and $n^{\log_b a} = n^2$.

  Since n^1 is polynomially smaller than n^2 by a factor of n (i.e., $\epsilon = 1$), this falls strictly under **Case 1**.

- **Step 4: Final Solution.**

  According to Case 1, $T(n) = \Theta(n^{\log_b a})$.

  Final Answer: $T(n) = \Theta(n^2)$

Q73. **Build an optimal Huffman Tree using the Huffman Coding Algorithm and generate the prefix code for the given letter frequencies.**

Answer:

Huffman Coding Algorithm Steps:

Huffman coding is a greedy algorithm used for data compression.

1. Create a leaf node for each character and add them to a Min-Priority Queue based on their frequencies (ascending order).
2. Extract the two nodes with the lowest frequencies.
3. Create a new internal node with a frequency equal to the sum of the two extracted nodes. Make the first extracted node the left child (edge = 0) and the second the right child (edge = 1).
4. Insert this new node back into the priority queue.
5. Repeat steps 2-4 until only one node (the Root) remains.

Example: Building the Optimal Huffman Tree

Let's use these frequencies: **A: 5, B: 9, C: 12, D: 13, E: 16, F: 45**

- **Initial Queue:** {A:5, B:9, C:12, D:13, E:16, F:45}
- **Merge 1:** Combine A(5) and B(9) → New Node N1(14).
  - *Queue:* {C:12, D:13, N1:14, E:16, F:45}
- **Merge 2:** Combine C(12) and D(13) → New Node N2(25).

- *Queue:* {N1:14, E:16, N2:25, F:45}
- **Merge 3:** Combine N1(14) and E(16) → New Node N3(30).
  - *Queue:* {N2:25, N3:30, F:45}
- **Merge 4:** Combine N2(25) and N3(30) → New Node N4(55).
  - *Queue:* {F:45, N4:55}
- **Final Merge:** Combine F(45) and N4(55) → Root Node (100).

Generating the Prefix Code:

To find the code for each character, trace the path from the Root (100) down to the specific leaf node. Add a 0 when you go left, and a 1 when you go right.

- **F:** 0
- **C:** 100
- **D:** 101
- **A:** 1100
- **B:** 1101
- **E:** 111

(This is an optimal prefix code because no character's code is a prefix of another character's code!)

Q74. **Trace Dijkstra's Single Source Shortest Path Algorithm step-by-step on a given graph, and write its time complexity and disadvantages.**

Answer:

Dijkstra's Algorithm Overview: Dijkstra's algorithm is a Greedy technique used to find the shortest path from a single source node to all other nodes in a graph with non-negative edge weights.

Step-by-Step Trace: Let's define a sample directed graph with 4 vertices: **A, B, C, D**.

- Source Node: **A**
- Edges & Weights: $(A \to B, 4)$, $(A \to C, 1)$, $(C \to B, 2)$, $(B \to D, 1)$, $(C \to D, 5)$.

We maintain a table. Initial distance to A is 0, and to all others is ∞ (Infinity).

Step	Unvisited Nodes	Chosen Node (Min Dist)	Dist to A	Dist to B	Dist to C	Dist to D
0	{A, B, C, D}	**A** (Dist 0)	**0**	∞	∞	∞
1	{B, C, D}	**C** (Dist 1)	0	**4** (via A)	**1** (via A)	∞
2	{B, D}	**B** (Dist 3)	0	**3** (via C)*	1	**6** (via C)
3	{D}	**D** (Dist 4)	0	3	1	**4** (via B)*
4	{ }	Algorithm Ends	0	3	1	4

Explanation of Updates:

- *Step 2:* From C, we can reach B with cost $1(\text{to C}) + 2(\text{C to B}) = 3$. Since $3 < 4$, we update B's distance.
- *Step 3:* From B, we can reach D with cost $3(\text{to B}) + 1(\text{B to D}) = 4$. Since $4 < 6$, we update D's distance.

Shortest Paths from Source A:

- A to B: Cost = 3 (Path: A → C → B)
- A to C: Cost = 1 (Path: A → C)
- A to D: Cost = 4 (Path: A → C → B → D)

Time Complexity:

- Using an Adjacency Matrix: $O(V^2)$
- Using an Adjacency List and a Min-Priority Queue (Min-Heap): $O(E \log V)$ (where V is vertices, E is edges).

Disadvantages:

- **Negative Weights:** It completely fails if the graph has negative weight edges. Once it marks a node as "visited", it assumes the shortest path to it is finalized and never re-evaluates it, which breaks if a negative edge could have reduced the cost further.
- **Blind Search:** It explores uniformly in all directions. (Algorithms like A* are better for directed target searching).

Q75. Solve step-by-step numericals for both the Fractional Knapsack Problem (Greedy Method) and the 0/1 Knapsack Problem (Dynamic Programming).

Answer:

To show the exact difference in execution, let's solve both using the exact same dataset.

- **Knapsack Capacity (W):** 5
- **Items ($n = 3$):**
  - Item 1: Weight = 1, Profit = 6
  - Item 2: Weight = 2, Profit = 10
  - Item 3: Weight = 3, Profit = 12

Fractional Knapsack (Greedy Method)

In Fractional Knapsack, we can break items into fractions. The greedy strategy is to sort items by their **Profit/Weight Ratio** in descending order.

1. **Calculate Ratios:**
   - Item 1: $6/1 = \mathbf{6}$
   - Item 2: $10/2 = \mathbf{5}$
   - Item 3: $12/3 = \mathbf{4}$

   (They are already sorted highest to lowest).
2. **Fill the Knapsack ($W = 5$):**
   - Take Item 1 (Weight 1): Profit = 6. Remaining Capacity = $5 - 1 = 4$.
   - Take Item 2 (Weight 2): Profit = 10. Remaining Capacity = $4 - 2 = 2$.
   - Item 3 has weight 3, but we only have capacity 2 left. We take a **fraction** $(2/3)$ of Item 3.
   - Fraction Profit = $(2/3) \times 12 = 8$. Remaining Capacity = 0.
3. **Total Profit:** $6 + 10 + 8 = \mathbf{24}$.

0/1 Knapsack (Dynamic Programming)

In 0/1 Knapsack, we *cannot* break items. We either take the whole item or leave it. We use a DP table where rows are items (i) and columns are capacities (w).

- Formula: $V[i][w] = \max(V[i-1][w], V[i-1][w - w_i] + p_i)$

Step-by-Step DP Table Trace:

i (Items) w (Capacity)	0	1	2	3	4	5
0 (No items)	0	0	0	0	0	0
1 (Item 1: w=1, p=6)	0	**6**	6	6	6	6
2 (Item 2: w=2, p=10)	0	6	**10**	**16**	16	16
3 (Item 3: w=3, p=12)	0	6	10	16	**18**	**22**

Explanation of key cells:

- V[2][3]: We have capacity 3 and items 1 & 2. We can fit both $(1 + 2 \leq 3)$, so profit is $6 + 10 = 16$.
- V[3][5]: Capacity 5, considering Item 3 (w=3, p=12).
  - Exclude Item 3: Profit = V[2][5] = 16.
  - Include Item 3: Profit = $12 + V[2][5 - 3] = 12 + V[2][2] = 12 + 10 = 22$.
  - Max is 22.

Total Profit: 22. *(Notice how DP gives a profit of 22 because we had to leave Item 1 behind to fit Item 2 and 3 perfectly into the weight of 5, whereas the Greedy fractional method gave 24).*

Q76. Construct a Minimum Cost Spanning Tree (MCST) using Prim's or Kruskal's algorithm on a weighted graph and state their time complexities.

Answer:

What is a Minimum Cost Spanning Tree (MCST)?

For a connected, undirected, and weighted graph, a Spanning Tree is a subgraph that connects all V vertices together using exactly V - 1 edges, without forming any cycles. A **Minimum Cost Spanning Tree (MCST)** is the spanning tree where the sum of all its edge weights is the absolute minimum possible.

Example Graph Setup:

Let us define a simple weighted graph with 4 vertices: **A, B, C, D**.

- Edges and Weights:
  - (A - B, Weight 1)
  - (B - C, Weight 2)
  - (C - D, Weight 3)
  - (A - D, Weight 4)
  - (A - C, Weight 5)

Solving using Kruskal's Algorithm:

Kruskal's algorithm is a greedy approach that sorts all edges by weight and adds them one by one to the tree, skipping any edge that forms a cycle.

- **Step 1: Sort all edges in ascending order of their weights.**
  1. (A - B): 1
  2. (B - C): 2
  3. (C - D): 3
  4. (A - D): 4
  5. (A - C): 5
- **Step 2: Pick edges one by one to form the MCST.** (We need exactly V - 1 = 3 edges).

Edge	Weight	Does it form a cycle?	Action Taken	Current MCST Edges
(A - B)	1	No	**Add**	{(A - B)}
(B - C)	2	No	**Add**	{(A - B), (B - C)}
(C - D)	3	No	**Add**	{(A - B), (B - C), (C - D)}

- **Step 3: Termination.** We now have 3 edges. The algorithm stops.
- **Final MCST Cost:** 1 + 2 + 3 = **6**.

Time Complexities:

- **Kruskal's Algorithm Complexity:** The dominant operation is sorting the edges. Using a good sorting algorithm and a Disjoint-Set (Union-Find) data structure, the time complexity is $O(E\log E)$ or $O(E\log V)$ (where E is edges and V is vertices).

- **Prim's Algorithm Complexity:** If implemented using an Adjacency List and a Min-Priority Queue, Prim's algorithm also has a time complexity of $O(E \log V)$.

Q77. Explain the principles of String Matching Algorithms, specifically the KMP Algorithm (including LPS array computation) and the Rabin-Karp Algorithm (Rolling Hash), with the help of examples.

Answer:

Knuth-Morris-Pratt (KMP) Algorithm & LPS Array

1. Principle of KMP:

The standard naive string matching algorithm is slow because whenever a mismatch occurs, it moves the text pointer backward and restarts the comparison. The KMP algorithm uses a pre-computed array called the **LPS (Longest Proper Prefix which is also a Suffix)** array. When a mismatch happens, KMP uses the LPS array to know exactly how many characters of the pattern have already matched, allowing it to shift the pattern forward safely **without ever moving the text pointer backward**.

2. LPS Array Computation Example:

Let the Pattern be **P = "ABABC"**.

- The LPS array stores the length of the longest proper prefix that matches a suffix for every sub-pattern.
- i = 0: Sub-pattern "A". No proper prefix/suffix. **LPS[0] = 0**.
- i = 1: Sub-pattern "AB". Prefix "A" ≠ Suffix "B". **LPS[1] = 0**.
- i = 2: Sub-pattern "ABA". Prefix "**A**" == Suffix "**A**". Length is 1. **LPS[2] = 1**.
- i = 3: Sub-pattern "ABAB". Prefix "**AB**" == Suffix "**AB**". Length is 2. **LPS[3] = 2**.
- i = 4: Sub-pattern "ABABC". Mismatch. It drops to 0. **LPS[4] = 0**.
- **Final LPS Array: [0, 0, 1, 2, 0]**

Rabin-Karp Algorithm & Rolling Hash

1. Principle of Rabin-Karp:

Instead of comparing characters one by one initially, Rabin-Karp calculates a numerical **Hash Value** for the pattern and a hash value for each sliding window of the text. If the hash values match, it performs a character-by-character check to confirm it's not a "hash collision".

2. The Rolling Hash Concept:

Calculating a new hash from scratch for every window takes too much time. A **Rolling Hash** calculates the hash of the *next* window in strictly **O(1) constant time**. It does this by:

1. Subtracting the value of the first character that just left the window.
2. Shifting the base multiplier.
3. Adding the value of the new character entering the window.

3. Rolling Hash Example:

- Let Text = 2359, Pattern = 35. Hash function = Integer Value $(mod 13)$.
- **Pattern Hash:** $35 (mod 13) = 9$.
- **Window 1 (23):** $23 (mod 13) = 10$. (Mismatch, $10 \neq 9$).
- **Window 2 (35) using Rolling Hash:** Instead of computing from scratch, we drop the leading 2, shift by multiplying by 10, and add the new 5.

  New Value $= (23 - 2 \times 10) \times 10 + 5 = (3) \times 10 + 5 = 35$.

  New Hash: $35 (mod 13) = 9$. (Match! 9 == 9. We then verify character by character).

Q78. Explain the working principle of the Floyd-Warshall Algorithm for All-Pairs Shortest Path, and generate the D^2 matrix for a given graph.

Answer:

Working Principle of Floyd-Warshall Algorithm:

The Floyd-Warshall algorithm uses the **Dynamic Programming** approach to find the shortest paths between *every single pair of vertices* in a directed weighted graph.

Its core principle is to iteratively consider every vertex in the graph as a potential "intermediate" connecting point to see if routing a path through it makes the journey shorter.

The recurrence relation it relies on is:

$$D^k[i][j] = \min\left(D^{k-1}[i][j], D^{k-1}[i][k] + D^{k-1}[k][j]\right)$$

("Is the currently known distance from i to j smaller, or is it shorter to go from i to k, and then from k to j?")

Generating the D^2 Matrix (Numerical Trace):

Let us define a standard 3-vertex directed graph to trace the matrices D^0, D^1, and finally D^2.

- Vertices: **1, 2, 3**.
- Direct Edges: (1 → 2, weight 8), (1 → 3, weight 5), (2 → 1, weight 3), (3 → 2, weight 2).
- If there is no direct edge, the distance is ∞. The distance from a node to itself is 0.

Step 1: Initial Matrix D^0 (Direct Edges Only)

| | 1 | 2 | 3 |

| :--- | :--- | :--- | :--- |

| **1** | 0 | 8 | 5 |

| **2** | 3 | 0 | ∞ |

| **3** | ∞ | 2 | 0 |

Step 2: Matrix D^1 (Allowing vertex 1 as an intermediate node)

We check if routing through node 1 makes any existing paths shorter.

- Path 2 → 3: Is (2 → 1) + (1 → 3) shorter than ∞?

  $3 + 5 = 8$. Since 8 < ∞, we update D[2][3] to 8.

- Path 3 → 2: Is (3 → 1) + (1 → 2) shorter than 2?

  ∞ + 8 = ∞. No update.

Current Matrix D^1:

| | 1 | 2 | 3 |

| :--- | :--- | :--- | :--- |

| **1** | 0 | 8 | 5 |

| **2** | 3 | 0 | **8** |

| **3** | ∞ | 2 | 0 |

Step 3: Matrix D^2 (Allowing vertex 2 as an intermediate node)

We now use the values from D^1 and check if routing through node 2 makes paths shorter.

- Path 1 → 3: Is (1 → 2) + (2 → 3) shorter than 5?

  8 + 8 = 16. Since 16 is not less than 5, no update.

- Path 3 → 1: Is (3 → 2) + (2 → 1) shorter than ∞?

  2 + 3 = 5. Since 5 < ∞, we update D[3][1] to 5.

Final Matrix D^2 (Your Answer):

| | 1 | 2 | 3 |

| :--- | :--- | :--- | :--- |

| **1** | 0 | 8 | 5 |

| **2** | 3 | 0 | 8 |

| **3** | **5** | 2 | 0 |

Q79. How is Matrix Chain Multiplication solved using Dynamic Programming? Solve a numerical problem to find the optimal parenthesization.

Answer:

Solving Matrix Chain Multiplication (MCM) using DP:

The goal of MCM is *not* to actually multiply the matrices, but to figure out the most efficient order (where to place the parentheses) to mathematically minimize the total number of scalar multiplications.

Multiplying a (p × q) matrix with a (q × r) matrix takes exactly p × q × r multiplications.

Using DP, we break the chain into smaller sub-chains. We build a cost table (m) to store the minimum multiplications and a split table (s) to store where we placed the optimal parentheses.

The recurrence relation used is:

$$m[i,j] = \min_{i \le k < j} \{m[i,k] + m[k+1,j] + P_{i-1}P_kP_j\}$$

(Where k is the point where we split the matrix chain).

Numerical Solution for Optimal Parenthesization:

Let us find the optimal parenthesization for 3 matrices with the dimension array P = [10, 100, 5, 50].

This means our matrices are:

- A_1 = 10 × 100
- A_2 = 100 × 5
- A_3 = 5 × 50

There are only two possible ways to group (parenthesize) these 3 matrices. Let's calculate the cost of both:

- **Option 1: $(A_1 \times A_2) \times A_3$**
  - First, multiply $A_1 \times A_2$: Cost = 10 × 100 × 5 = 5000. The resulting matrix is (10 × 5).
  - Next, multiply the Result × A_3: Cost = 10 × 5 × 50 = 2500.
  - **Total Cost:** 5000 + 2500 = 7500 multiplications.
- **Option 2: $A_1 \times (A_2 \times A_3)$**
  - First, multiply $A_2 \times A_3$: Cost = 100 × 5 × 50 = 25000. The resulting matrix is (100 × 50).
  - Next, multiply A_1 × Result: Cost = 10 × 100 × 50 = 50000.
  - **Total Cost:** 25000 + 50000 = 75000 multiplications.

Conclusion: Comparing the two, 7500 is massively smaller than 75000. Therefore, the optimal parenthesization is **$(A_1 \times A_2) \times A_3$)**, and the minimum number of scalar multiplications required is **7500**.

Q80. Trace Divide & Conquer based Sorting Algorithms (Merge Sort or Quick Sort) on an array and derive their best and worst-case time complexities.

Answer:

Tracing Merge Sort (Divide & Conquer):

Merge Sort strictly follows the Divide & Conquer approach by continuously dividing the array into two equal halves until single elements remain, and then merging them back together in sorted order.

Let's trace it on the array: [38, 27, 43, 3, 9, 82, 10]

- **Phase 1: Divide (Splitting)**
  - Split 1: [38, 27, 43, 3] and [9, 82, 10]
  - Split 2: [38, 27], [43, 3] and [9, 82], [10]
  - Split 3: [38], [27], [43], [3], [9], [82], [10] *(All are now single, sorted elements)*
- **Phase 2: Conquer & Combine (Merging)**
  - Merge 1: [27, 38], [3, 43] and [9, 82], [10]
  - Merge 2: [3, 27, 38, 43] and [9, 10, 82]
  - Final Merge: Compare elements one by one from both halves:

    3 < 9 → [3]

    9 < 27 → [3, 9]

    10 < 27 → [3, 9, 10]

    ...

  - **Final Sorted Array: [3, 9, 10, 27, 38, 43, 82]**

Deriving Time Complexities:

A. Merge Sort:

- **Recurrence Relation:** T(n) = 2T(n/2) + O(n) (Divides into 2 subproblems of half size, merging takes linear time).
- Using the Master Theorem, a=2, b=2, f(n)=n. Since $n^{\log_2 2} = n$, this is Case 2.
- **Best, Average, and Worst Case:** $O(n \log n)$ (It always divides the array exactly in half, regardless of the initial order).

B. Quick Sort:

- **Best Case:** Occurs when the pivot perfectly divides the array into two equal halves.
  - Recurrence: T(n) = 2T(n/2) + O(n)
  - Complexity: $O(n \log n)$

- **Worst Case:** Occurs when the array is already sorted (or reverse sorted) and the smallest/largest element is picked as the pivot, dividing the array of size n into sizes 0 and n-1.
    - Recurrence: T(n) = T(n-1) + O(n)
    - Solving this yields the sum of the first n integers.
    - Complexity: $O(n^2)$

Q81. Differentiate between the Complexity Classes: P, NP, NP-Hard, and NP-Complete, and explain the Cook-Levin theorem (CNF-SAT problem).

Answer:

Differences Between Complexity Classes:

In theoretical computer science, decision problems (problems with a "Yes" or "No" answer) are categorized based on how fast a computer can solve them or verify them.

Complexity Class	Definition & Characteristics	Example
P (Polynomial Time)	Problems that can be **solved** quickly (in polynomial time like $O(n^2)$) by a standard deterministic computer.	Sorting, Searching, Shortest Path.
NP (Non-Deterministic Polynomial)	Problems whose proposed solutions can be **verified** quickly. If someone gives you an answer, you can check if it's correct in polynomial time. (Note: All P problems are also in NP).	Sudoku, Traveling Salesperson Problem (TSP).
NP-Complete	The absolute **hardest problems inside NP**. If you can find a fast algorithm to solve just one NP-Complete problem, you can solve *all* NP problems quickly (proving P = NP).	Boolean Satisfiability (SAT), Graph Coloring.
NP-Hard	Problems that are at least as hard as NP-Complete problems, but they **do not have to be in NP** (meaning you might not even be able to verify their answers quickly).	The Halting Problem, finding the absolute best move in Chess.

Cook-Levin Theorem & CNF-SAT Problem:

- **The Theorem:** The Cook-Levin Theorem was a massive breakthrough because it proved that the **Boolean Satisfiability Problem (SAT)** is the world's very first **NP-Complete** problem. It proved that every single problem in the NP class can be mathematically translated (reduced) into a SAT problem in polynomial time.
- **What is CNF-SAT?**

  CNF (Conjunctive Normal Form) is a way of writing boolean logic formulas as an "AND of ORs".

    - *Example Formula:* $F = (x_1 \vee x_2) \wedge (\neg x_1 \vee x_3)$
    - The problem asks: "Is there any combination of TRUE/FALSE for the variables x_1, x_2, x_3 that makes the whole formula evaluate to TRUE?"
    - If you guess $x_1 =$ TRUE, $x_2 =$ FALSE, $x_3 =$ TRUE, the formula evaluates to TRUE. Verifying this guess is incredibly fast (hence it is in NP), but finding the correct combination from scratch for a formula with thousands of variables takes exponential time.

QUESTIONS LIST

1. What is an algorithm? What are its desirable characteristics? Explain characteristics of an algorithm with the help of an example.
2. What are the building blocks of an algorithm? Explain how to judge an algorithm, whether it is efficient or not?
3. What is Euclid's algorithm to find GCD of two given integers? Write the steps involved in finding GCD of (a, b) using Euclid's algorithm.
4. What are Optimization and Decision problems? Give an example of each. Formulate Traveling salesperson problem & Graph coloring problem as optimization and decision problems.
5. "The best-case analysis is not as important as the worst-case analysis of an algorithm." Yes or No. Justify your answer with the help of an example.
6. What are asymptotic notations? Explain any two asymptotic notations with suitable example for each.
7. Write a mathematical definition of O (Big Oh). Assume that the function f(n) = 2n^2 + 3n + 1. Show that f(n) = O(n^2).
8. Write a mathematical definition of Big Omega (Ω). For the functions defined by f(n) = 3n^3 + 2n^2 + 1 and g(n) = 2n^2 + 3, verify that f(n) = \Omega(g(n)).
9. What are Big 'O' and Big 'Ω' notations? Explain with the help of a representative diagram.
10. Arrange the following growth rates in increasing order: O(n^2), O(2^n), O(n \log n), O(2!), O(1), O(\log n).
11. Whether the following is in correct order? 1, \log n, n, n \log n, n^2, 2^n, n!
12. Calculate the time complexity of the following program fragments using Big Oh notation:
(i) For (i=0; i<n; i++) a[i]=0;
(ii) For (i=1; i<=n; i=i*2) { x=x+i; }
(iii) for(i=0; i<n; i++) for (j=0; j<n; j++) A[i]=A[i]+A[j];
13. List one algorithm each for the following time complexities: (i) O(m \log n) (ii) O(\log n) (iii) O(n^2).
14. Using mathematical induction, prove that the sum of first 'n' positive integers is (n(n+1)/2).
15. Use mathematical induction to prove that: \sum i^2 = \frac{n(n+1)(2n+1)}{6}.
16. Prove that for all non-negative integers n: 2^0 + 2^1 + 2^2 + \dots + 2^n = 2^{n+1} - 1.
17. What is polynomial evaluation? What are its methods of evaluation? Evaluate P(x) = 3x^2 + 5x + 6 using Horner's rule at x=3.
18. Apply Horner's method for evaluating a polynomial expression p(x) = 6x^6 + 5x^5 + 4x^4 - 3x^3 + 8x - 7 at x = 3. Calculate: (i) How many times will the loop execute? (ii) What will be the total number of multiplication and addition operations?
19. Compute x^{283} by using left-to-right binary exponentiation.
20. Give a recursive function to find the height of a binary tree. What is the running time of this algorithm?
21. Explain the use of master method. Write and interpret all the three cases of the master method to solve recurrence relation problem.
22. Apply a master method to give the tight asymptotic bounds of the following recurrences: (i) T(n) = 4T(n/2) + n^2 (ii) T(n) = 9T(n/3) + n.
23. Define the substitution method to solve a recurrence relation. Solve the following recurrence relation using substitution method: T(n) = 2T(n/2) + n.
24. Define a recurrence relation of QuickSort algorithm and solve it using a recurrence tree.
25. Solve the given recurrence relation using recursion tree method: T(n) = 2T(n/2) + n.
26. Write and explain linear search algorithm. Mention best case, worst case and average case scenarios of linear search.
27. Write and explain binary search algorithm with a suitable example. How many comparisons are needed for a binary search in a set of 512 elements?
28. Write recursive binary search algorithms and analyse its complexity in worst case scenario.
29. Explain Quick sort algorithm using divide and conquer approach. Apply the partition procedure of Quicksort algorithm to the following array: [35, 10, 40, 5, 60, 25, 45, 15].
30. Explain merge sort algorithm using divide and conquer approach. Also mention its best case and worst case time complexities. Apply the same to sort the array of elements: [7, 15, 5, 9, 20, 18, 25, 17, 11].
31. Multiply the following two matrices using Strassen's algorithm.
32. Multiply the following two numbers using Karatsuba's algorithm.
33. Give a divide and conquer based algorithm to find the i^{th} smallest element in an array of size n: Trace your algorithm to find 3rd smallest in the array: A = [10, 2, 5, 15, 50, 6, 20, 25].
34. Write Bubble Sort Algorithm. Using bubble sort, sort the following sequence: 20, 15, 25, 10, 8, 38, 27, 11. Find the total number of comparisons.
35. Sort the following sequence of numbers, using selection sort. Also find the number of comparisons and copy operations required by the algorithm in sorting this list: 28, 13, 12, 28, 35, 11, 15, 9, 36.

36. Explain any three of the following terms with the help of a suitable diagram: (i) Subgraph (ii) Connected graph (iii) Adjacency matrix (iv) Directed acyclic graph.
37. What are strongly connected components? Explain how adjacency matrix and adjacency list are created for a connected graph with the help of a suitable diagram.
38. Explain Depth First Search (DFS) algorithm with a suitable example. Apply DFS to the complete graph on four vertices. List the vertices in the order they would be visited.
39. Apply the DFS algorithm to the following graph with the starting vertex v_1. List the order in which vertices will be visited. Show the complexity analysis if a graph is represented through (i) Adjacency list, and (ii) Adjacency matrix.
40. Explain Breadth First Search (BFS) algorithm with a suitable example. Define a BFS tree. Give the breath first traversal for the undirected graph starting from vertex 'a'. Also give any three applications of DFS.
41. Define topological ordering of a graph. Write the algorithm to find topological ordering of the following graph. Calculate the complexity of the algorithm.
42. What is the similarity between Dijkstra's single source shortest path and Prim's minimum cost spanning tree algorithms?
43. Describe the most commonly used data structure for implementing Dijkstra single source shortest path algorithm. What are the disadvantages of using Dijkstra's algorithm? List out any four.
44. Prove that "Subpaths of the shortest path in a single source shortest path algorithm are also the shortest paths."
45. Write Dijkstra's algorithm to find the shortest path in a graph. Apply Dijkstra's algorithm on the following graph.
46. What is the significant feature of Bellman-Ford's algorithm which is not supported in Dijkstra's algorithm?
47. Write and explain the procedure to find a solution to maximum bipartite matching problem with the help of an example. Write and explain pseudocode for Ford-Fulkerson's algorithm.
48. What is Greedy approach for problem solving? How does a Greedy algorithm work? Write the activities performed in Greedy method.
49. Differentiate between greedy approach and dynamic approach to solve an optimization problem.
50. What is a Minimum Cost Spanning Tree (MCST)? Write Generic MCST algorithm.
51. Write Prim's algorithm for finding Minimum Spanning Tree and find its time complexity. Also find MST of the following graph using Prim's algorithm.
52. Write Kruskal's algorithm to find minimum cost spanning tree of the following graph. Show the step by step execution of Kruskal's algorithm.
53. Define a fractional knapsack problem as an optimization problem. Write a greedy method to find an optimal solution to the problem. Show the complexity of the algorithm.
54. What is Task Scheduling algorithm? Write pseudo code for task scheduling algorithm. Describe the task scheduling algorithm as an optimization problem and calculate its complexity. Apply the task scheduling algorithm to minimize the total amount of time spent in the system.
55. What is Huffman coding? Write the steps for building the Huffman tree with an example. Construct an optimal Huffman tree and Huffman code for each character for the given set of frequencies.
56. What is Dynamic Programming approach of problem solving? Write the steps involved in dynamic programming. Explain the principle of optimality in dynamic programming, with the help of an example.
57. Explain the 0/1 Knapsack problem. Solve the 0/1 Knapsack problem using Dynamic Programming.
58. What is Matrix chain multiplication problem? Explain how dynamic programming can be used to solve matrix chain multiplication. Find an optimal parenthesization of a matrix-chain product. List all the different orders in which we can multiply five matrices M1, M2, M3, M4, M5.
59. What is all pair shortest path problem? Write and explain Floyd Warshall algorithm for shortest paths with the help of a diagram. Explain the working principle of Floyd-Warshall's algorithm. Apply Floyd Warshall's algorithm and show the matrix D^2 of the graph.
60. A binomial coefficient is defined by a recurrence relation. Write a recursive function to generate C(n, k).
61. What is string matching problem? What is string matching algorithm? Derive its best case time complexity.
62. Explain the naïve string matching algorithm and derive its worst case complexity. What is its drawback? What will be the maximum valid shifts of a pattern in the text in the following example?
63. Explain Knuth Morris Pratt (KMP) algorithm of string matching with a suitable example. Explain the process of building LPS array for a pattern 'P'.
64. Describe the basic principle of KMP algorithm for string matching. What is its advantage compared to Naive and Rabin-Karp's algorithm for string matching? Calculate the time complexity of KMP algorithm.
65. Explain the concept of rolling hash function applied in Rabin-Karp algorithm for string matching problem with the help of an example.
66. Explain P, NP and NP-complete class of problems with appropriate examples of each class. Differentiate between NP-Hard and NP-Complete problems.
67. Explain the concept of non-deterministic algorithm with the help of an example. List the problems which belong to non-deterministic class of complexity.
68. Explain Cook-Levin's theorem on CNF-Satisfiability problem, with the help of an example.
69. Design a state space tree for the given subset sum problem. S=4,6,7,8, W = 8.
70. What are the key features of combinatorial problems? Describe and formulate three combinatorial problems.
71. Short notes on: Deterministic algorithms vs. Non-deterministic, CLIQUE and vertex cover problem, Backtracking problem with example, Tractable vs. Intractable problems, Approximation algorithms.
72. Explain the three cases of the Master Theorem and solve the recurrence relation (e.g., T(n) = 9T(n/3) + n).

73. Build an optimal Huffman Tree using the Huffman Coding Algorithm and generate the prefix code for the given letter frequencies.

74. Trace Dijkstra's Single Source Shortest Path Algorithm step-by-step on a given graph, and write its time complexity and disadvantages.

75. Solve step-by-step numericals for both the Fractional Knapsack Problem (Greedy Method) and the 0/1 Knapsack Problem (Dynamic Programming).

76. Construct a Minimum Cost Spanning Tree (MCST) using Prim's or Kruskal's algorithm on a weighted graph and state their time complexities.

77. Explain the principles of String Matching Algorithms, specifically the KMP Algorithm (including LPS array computation) and the Rabin-Karp Algorithm (Rolling Hash), with the help of examples.

78. Explain the working principle of the Floyd-Warshall Algorithm for All-Pairs Shortest Path, and generate the D^2 matrix for a given graph.

79. How is Matrix Chain Multiplication solved using Dynamic Programming? Solve a numerical problem to find the optimal parenthesization.

80. Trace Divide & Conquer based Sorting Algorithms (Merge Sort or Quick Sort) on an array and derive their best and worst-case time complexities.

81. Differentiate between the Complexity Classes: P, NP, NP-Hard, and NP-Complete, and explain the Cook-Levin theorem (CNF-SAT problem).